



 Latest updates: <https://dl.acm.org/doi/10.1145/3793532>

SURVEY

Confidential Computing on Heterogeneous CPU-GPU Systems: Survey and Future Directions

QIFAN WANG, Durham University, Durham, County Durham, U.K.

DAVID OSWALD, Durham University, Durham, County Durham, U.K.

Open Access Support provided by:

Durham University



PDF Download
3793532.pdf
26 March 2026
Total Citations: 0
Total Downloads: 663

Published: 25 February 2026
Online AM: 26 January 2026
Accepted: 19 January 2026
Revised: 11 June 2025
Received: 03 September 2024

[Citation in BibTeX format](#)

Confidential Computing on Heterogeneous CPU-GPU Systems: Survey and Future Directions

QIFAN WANG, Durham University, Durham, United Kingdom of Great Britain and Northern Ireland

DAVID OSWALD, Durham University, Durham, United Kingdom of Great Britain and Northern Ireland

In recent years, the widespread informatization and rapid data explosion have increased the demand for high-performance heterogeneous systems that integrate multiple computing cores such as Central Processing Units (CPUs), Graphics Processing Units (GPUs), Application Specific Integrated Circuits (ASICs), and Field Programmable Gate Arrays (FPGAs). The combination of CPU and GPU is particularly popular due to its versatility. However, these heterogeneous systems face significant security and privacy risks. Advances in privacy-preserving techniques, especially hardware-based Trusted Execution Environments (TEEs), offer effective protection for GPU applications. Nonetheless, the potential security risks involved in extending TEE to GPU in heterogeneous systems remain uncertain and need further investigation. To investigate these risks in depth, we study the existing popular GPU TEE designs and summarize and compare their key implications. Additionally, we review existing powerful attacks on GPU and traditional TEE deployed on CPUs, along with the efforts to mitigate these threats. We identify potential attack surfaces introduced by GPU TEE and provide insights into key considerations for designing secure GPU TEE. This survey is timely as new TEE for heterogeneous systems, particularly GPU, are being developed, highlighting the need to understand potential security threats and build both efficient and secure systems.

CCS Concepts: • **General and reference** → **Surveys and overviews**; • **Computer systems organization** → *Heterogeneous (hybrid) systems*; • **Security and privacy** → **Hardware attacks and countermeasures**; **Trusted computing**;

Additional Key Words and Phrases: Confidential computing, trusted execution environment on GPU, hardware security, GPU attacks

ACM Reference Format:

Qifan Wang and David Oswald. 2026. Confidential Computing on Heterogeneous CPU-GPU Systems: Survey and Future Directions. *ACM Comput. Surv.* 58, 9, Article 231 (February 2026), 35 pages. <https://doi.org/10.1145/3793532>

1 Introduction

Heterogeneous computing systems have become a cornerstone of modern technology, driven by the need for enhanced energy efficiency and performance across a wide range of applications. From the immense computational power of supercomputers and high-performance clusters to the everyday use of smartphones and laptops, diverse computational architectures are now integrated to achieve

This research is partially funded by the Engineering and Physical Sciences Research Council (EPSRC) under grants EP/X03738X/1, EP/V000454/1, and EP/R012598/1. The results feed into DsbDtech.

Authors' Contact Information: Qifan Wang (corresponding author), Durham University, Durham, England, United Kingdom of Great Britain and Northern Ireland; e-mail: qifan.wang2@durham.ac.uk; David Oswald, Durham University, Durham, England, United Kingdom of Great Britain and Northern Ireland; e-mail: david.f.oswald@durham.ac.uk.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2026 Copyright held by the owner/author(s).

ACM 0360-0300/2026/02-ART231

<https://doi.org/10.1145/3793532>

the best balance between performance and power consumption [68, 97]. Heterogeneous computing refers to systems that leverage multiple types of computing cores, including **Central Processing Units (CPUs)**, **Graphics Processing Units (GPUs)**, **Application Specific Integrated Circuits (ASICs)**, **Field Programmable Gate Arrays (FPGAs)**, and **Neural Processing Units (NPU)s**. Although the concept of heterogeneous computing has been explored for over two decades, recent advancements in accelerators like GPUs, coupled with the explosive growth of **Machine Learning (ML)**, have significantly transformed these systems. In particular, since the mid-2000s, research has demonstrated that GPUs can accelerate the training of **Convolutional Neural Networks (CNN)** and other popular ML models by 10–30× compared with traditional CPUs [127, 131]. This acceleration makes heterogeneous computing systems using GPUs indispensable in today’s ML infrastructure, providing critical support for the rapid development and deployment of **Artificial Intelligence (AI)** technologies. While heterogeneous computing systems deliver exceptional performance and power efficiency, they also introduce new challenges. The complex architectures of these systems can create potential vulnerabilities and security risks, especially for the tasks they execute, such as ML applications. Ensuring the security and reliability of these systems is essential as they continue to evolve and expand their role in the computing landscape.

To ensure the security and confidentiality of heterogeneous computing systems, researchers have focused on adapting privacy-preserving techniques traditionally used on CPUs to accelerators like GPUs. These techniques include cryptographic methods such as **Homomorphic Encryption (HE)** [35, 96, 108] and Secure **Multi-Party Computation (MPC)** [60, 96, 108, 131, 148], as well as **Trusted Execution Environments (TEEs)** tailored for accelerators like GPUs [26, 57, 58, 61, 75, 92, 109, 140, 141, 154, 162]. TEEs offer certain advantages over cryptographic methods, particularly in performance and ease of deployment, providing a robust hardware-based security mechanism that is relatively straightforward to implement and manage. However, in contrast to cryptographic techniques, the security of TEEs relies on trust in the manufacturer and is difficult to analyse and prove rigorously.

Recent research has explored extending TEEs to accelerators like GPUs, both for x86 [57, 58, 75, 92, 109, 140, 154, 162] and Arm [26, 61, 141] architectures. Most of these approaches involve modifying existing CPU TEEs (especially process-based TEEs like Intel **Software Guard Extensions (SGX)** and Arm **TrustZone (TrustZone)**) to secure the GPU software stack, such as GPU drivers, and proposing strategies to ensure the isolation and confidentiality of GPU kernel executions. Apart from this, NVIDIA has introduced a pioneering commercial platform in its Hopper architecture [109], which supports confidential computing through features like memory encryption and isolated execution contexts. However, the security threats associated with extending TEEs across both CPUs and accelerators in heterogeneous computing systems remain uncertain and require further investigation.

In this survey, we delve into the potential vulnerabilities in current TEE designs on accelerators, with a specific emphasis on GPUs (referred to as *GPU TEEs*). To this end, we begin by reviewing and classifying research efforts dedicated to creating GPU TEE prototypes. These solutions target various aspects: some aim at securing the GPU software stack, others focus on safeguarding the confidentiality of data processed on the GPU, and some provide robust isolation mechanisms. Consequently, each design presents different attack surfaces. To identify these potential attack surfaces, we categorize and analyze existing attacks on GPUs, including architectural/logical attacks, microarchitectural side-channel and covert-channel attacks, physical side-channel attacks, and fault injection. The unique architecture and accessibility of GPUs within heterogeneous computing systems pose significant differences compared with traditional CPUs, which have historically been the primary targets of these attacks. Additionally, we also review existing attacks against CPU TEEs

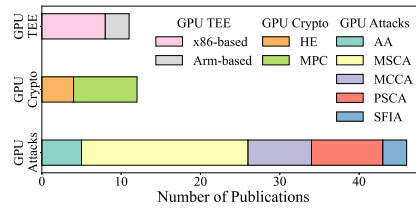


Fig. 1. Current state of security research on heterogeneous computing systems with GPU (from the past 15 years). *GPU Crypto* refers to papers that protect the privacy of GPU-based applications using cryptographic techniques such as HE and MPC. *GPU attacks* refer to architectural (AA), microarchitectural side-channel/covert-channel (MSCA/MCCA), physical side-channel (PSCA), and software-based fault injection attacks (SFIA).

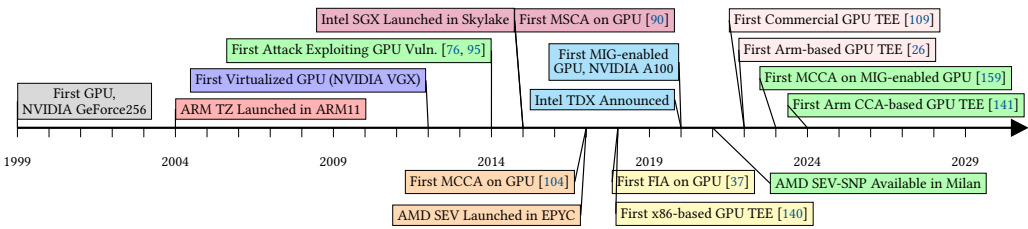


Fig. 2. Timeline of significant events in secure heterogeneous computing systems.

such as Intel SGX, Arm TZ, and AMD **Secure Encrypted Virtualization (SEV)**, and investigate their implications for code and data running on GPU TEEs. Finally, by synthesizing these insights, we discuss the potential attack surfaces in GPU TEEs exposed by these powerful attack vectors.

As shown in Figure 1 and Figure 2, the evolution of GPUs has led to the development of increasingly high-performance architectures. Consequently, research has increasingly focused on investigating potential attack surfaces in GPUs by exploring new architectural vulnerabilities. Likewise, we expect the potential security threats of recently emerging GPU TEEs to receive growing attention. We believe that this survey provides critical insights into key considerations for designing GPU TEEs, the primary workflows of existing attacks targeting heterogeneous computing systems, and the potential attack surface introduced for both CPU TEEs and GPUs. This understanding can guide future research into developing defenses for secure high-performance computing on heterogeneous systems and enhance confidential computing on GPU TEEs by refining the **Trusted Computing Base (TCB)**. Moreover, we believe our work can inspire the creation of more GPU TEE prototypes that are well-suited for various architectures and real-world applications.

2 Background

In this section, we introduce the basics of GPUs, common attack types, and the most existing popular TEE designs.

2.1 General Purpose Graphics Processing Units

General Purpose GPUs (GPGPUs) are specialized accelerators designed to handle diverse compute-intensive workloads beyond traditional graphics rendering. With their robust parallel computing capabilities, these GPUs are ideal for applications such as data analysis, ML, cryptography, and so on. Typically, a GPGPU consists of multiple processing units (i.e., CUDA cores in NVIDIA GPUs and shader cores and execution units in Arm GPUs) capable of executing thousands of threads simultaneously. Aside from GPUs, there are several prominent accelerators designed to

Table 1. Summary of Existing Popular GPUs

Accelerator	Year	Features	Virtualization	Shared Memory Hierarchy			Memory Size		
				None	L2/L3 Cache	DRAM	L1	L2	DRAM
Intel	HD/Iris/UHD/Xe	2010-2020	Integrated graphics	GVT	✓	✓	Shared	Shared	Shared
NVIDIA	P100	2016	HBM2	vGPU/MPS	✓		64 KB	4MB	16 GB
	V100	2017	Optimized SM for ML	vGPU/MPS	✓		128 KB	6MB	16 GB
	A100	2020	3G Tensor Cores and NVLink	vGPU/MPS/MIG	✓		192 KB	40 MB	40 GB
	H100	2022	Confidential computing	vGPU/MPS/MIG	✓		256 KB	50 MB	80 GB
AMD	Radeon RX 580	2017	FreeSync, CrossFire	N/A	✓		16 KB	2 MB	4/8 GB
	Radeon RX 6900	2020	Infinity Cache, SAM	N/A	✓		80 KB	4 MB	16 GB
Arm	Mali-T624 GPU	2013	Unified Shader Architecture	N/A		✓	16 KB	256 KB	Shared
Qualcomm	Adreno GPU 800	2020	AI Acceleration	N/A		✓	32-64 KB	256 KB-1 MB	Shared

In **Virtualization**, vGPU, GVT, MPS, and MIG denote Virtual Graphics Processing Unit, Graphics Virtualization Technology, Multi-Process Service, and Multi-Instance GPU. **Memory Sharing** denotes whether a level of the memory hierarchy is shared between the CPU and the accelerator.

boost performance for specific computational tasks. These include FPGAs, ASICs, and NPUs. In this survey, our primary focus is on heterogeneous systems utilizing GPUs.

With the evolution of GPUs, vendors like Intel, NVIDIA, AMD, Arm, Qualcomm, and others have introduced GPUs with distinct architectural features and capabilities. Different architectural features, such as the memory hierarchy, may introduce different attack surfaces on GPUs. Existing widely used GPUs, detailed in Table 1, vary in their architectural designs, particularly concerning their memory hierarchy, which can affect their vulnerability to certain types of attacks (e.g., microarchitectural attacks on integrated and discrete GPUs in Section 5). To provide a comprehensive overview of these GPUs, we consider the aspects in the following sections.

Integrated/Discrete GPU. Integrated GPUs, e.g., Intel HD/Iris/UHD/Xe Graphics, Arm, and Qualcomm GPUs in Table 1, are integrated on the same die or package as the CPU. These GPUs share the system’s memory with the CPU. Discrete GPUs are standalone and dedicated graphics processing units separate from the CPU. They have their dedicated memory, which is distinct from the system memory (e.g., NVIDIA and AMD GPUs in Table 1). NVIDIA GPUs have long supported host memory access via **Peripheral Component Interconnect Express (PCIe)** and Unified Memory, but recent architectures like the H100—especially in Grace Hopper configurations—enable tighter CPU-GPU integration with coherent memory sharing over NVLink-C2C [110].

GPU virtualization. *Graphics Virtualization Technology (GVT)*, a.k.a. Intel GVT, enables a single physical GPU to be shared among multiple **Virtual Machines (VMs)** with dedicated GPU resources. *Multi-Process Service (MPS)* from NVIDIA manages multiple CUDA applications concurrently on a single GPU but lacks the address space isolation of true GPU virtualization, making it unsuitable for multi-tenant systems [159]. NVIDIA’s *Virtual Graphics Processing Unit (vGPU)*, unlike MPS, ensures separate GPU address spaces and execution contexts. *Multi-Instance GPU (MIG)*, introduced in NVIDIA’s A100 GPU architecture, partitions a single physical GPU into multiple instances. Each MIG instance operates as a separate GPU with dedicated resources, including compute cores, memory, and cache.

Open-source status of GPU source code. The availability of GPU source code, including drivers and low-level architectural details, varies depending on the vendor’s policies and the specific GPU architecture. For instance, vendors like AMD (e.g., AMD’s ROCm [52]) and Intel (Mesa 3D Graphics [56]) offer open-source GPU drivers for specific platforms. However, NVIDIA, ARM, and Qualcomm typically provide proprietary drivers, limiting developers’ access to design details. As a result, many research efforts on GPU attacks often rely on reverse engineering to acquire details like **Translation Lookaside Buffer (TLB)** levels, warp schedulers, and more (see Section 5).

2.2 Types of GPU Attacks

In recent decades, a range of attacks has emerged as significant threats to modern CPU-based computing systems. As the use of GPUs becomes more prevalent, there is growing interest in exploring potential security vulnerabilities that extend from the CPU to the GPU. In this survey, we detail various attack vectors, including architectural/logical attacks, microarchitectural side-channel and covert-channel attacks, physical side-channel attacks, and fault injection.

2.2.1 Architectural Attacks. These attacks exploit vulnerabilities in the design and configuration of hardware components. This includes misuse of GPU or processor features, privilege escalation, and weak memory isolation. Attackers may target processing units (e.g., processors, hardware accelerators, and co-processors), the memory subsystem, and communication components responsible for interfacing with other system elements to bypass protections or leak sensitive data [39]. *Logical attacks*, a subset of architectural attacks, exploit flaws in the intended logic or functional behavior of the hardware. These may involve undocumented instructions, incorrect permission checks, or logic bugs that allow attackers to alter control flow or data access patterns.

2.2.2 Microarchitectural Side-Channel and Covert-Channel Attacks. Microarchitectural side-channel attacks exploit unintended behaviours of a processor's microarchitecture to extract sensitive information. They target components like cache behaviour, branch prediction, or speculative execution. Initially demonstrated on CPUs, these attacks exploit various optimization structures such as caches [42, 43, 88, 112] and branch predictors [32, 33]. Recently, a new category of microarchitectural attacks, known as transient-execution attacks [151], has emerged on CPUs. Examples include Meltdown [86], Spectre [70], and **Microarchitectural Data Sampling (MDS)** [117, 139], which rely on out-of-order and transient (speculative) execution in modern CPUs. Unlike physical side-channel attacks, microarchitectural attacks necessitate an understanding of the specific microarchitecture and its inherent vulnerabilities.

A microarchitectural covert-channel attack is a subtype of a side-channel attack. Covert channels involve two processes establishing an unauthorized communication pathway through an unintended channel to leak information, making them relevant in cloud computing environments [102, 160]. Due to imperfections in isolation and shared resource usage, malicious processes can exploit such resources to establish a communication channel with the victim process. The covert channel typically involves a sender, known as the "trojan", and a receiver, often referred to as the "spy". Through covert channels, the trojan can transmit sensitive information or leak data to the spy [2].

2.2.3 Physical Side-Channel Attacks. These passive attacks exploit the physical characteristics of a computing device, such as power consumption, **Electro-Magnetic (EM)**, or timing information, to gather insights into the device's operations. Examples include attacks such as **Differential Power Analysis (DPA)**, which exploits fluctuations in power consumption to extract cryptographic keys or other sensitive data. Unlike microarchitectural attacks, physical side-channel attacks do not always demand knowledge of the device's internal architecture or vulnerabilities. Instead, they use the unintentional leakage of information through physical signals and behaviour.

2.2.4 Fault Injection Attack. Fault injection attacks involve actively introducing faults or errors into a system to alter its behavior or extract sensitive information. These attacks can be generally categorized into non-invasive, semi-invasive, and invasive [129]. Non-invasive attacks allow adversaries to use methods such as clock and voltage glitching, overheating, EM faults, software faults, and remote hardware faults, with either physical or non-physical access. Semi-invasive attacks require the (partial) decapsulation of the target chip for techniques like laser and optical fault injection, while invasive attacks involve more direct methods, such as micro-probing and IC

modification, both of which necessitate deep physical access. In this work, we primarily focus on non-invasive fault injection attacks.

2.2.5 Comparison of Attacks on CPUs and GPUs. While attacks on CPU and GPU share some fundamental principles—such as exploiting timing variations, memory leakage, or shared resources—their feasibility, targeted attack surfaces, and required attacker capabilities diverge significantly due to underlying architectural and operational differences. CPUs generally enforce stricter privilege separation and memory isolation, making attacks like residual memory leakage less effective. In contrast, GPUs have historically lacked such safeguards, making them more vulnerable to memory remanence and weak access control. For microarchitectural attacks, GPUs introduce new leakage vectors due to their massive parallelism and SIMT execution model, allowing attackers to exploit fine-grained resource contention (e.g., warps, shared caches) at scale. GPUs also expose performance counters and **Application Programming Interfaces (APIs)** more openly than CPUs, increasing their attack surface. In physical side-channel attacks, GPUs tend to emit more observable signals due to their higher power usage and bandwidth, and their close coupling with CPUs introduces additional leakage through shared interconnects or **Memory-Mapped I/O (MMIO)**. For fault injection, while CPUs are often targeted through physical glitching, GPU attacks frequently exploit software-exposed features like **Dynamic Voltage and Frequency Scaling (DVFS)** or power-state manipulation to induce faults during kernel execution. ML workloads on GPUs are resilient to random faults but remain vulnerable to precise, targeted injections. Overall, while CPUs benefit from more mature defense mechanisms, GPU security is still evolving. Adapting proven CPU defenses to GPU-specific designs and threat models remains an important direction for future research.

2.3 TEE

Most existing GPU TEEs rely on a CPU TEE to protect the GPU drivers on the CPU host, which can be compromised by the adversary. This section introduces CPU TEEs, including process-based TEEs such as Intel SGX and Arm TZ, which provide isolation at the process level, and virtualization-based TEEs like AMD SEV, Arm **Confidential Compute Architecture (CCA)**, and Intel **Trust Domain Extensions (TDX)**, which offer isolation at the VM level.

2.3.1 Intel SGX Overview. SGX offers a secure and isolated environment, termed *enclave*, for sensitive workloads without the need to rely on the security of the untrusted host **Operating System (OS)**. SGX ensures the authenticity of executed code, maintains the integrity of runtime states (e.g., CPU registers, memory, and sensitive I/O), and preserves the confidentiality of enclave code, data, and runtime states stored in persistent memory through a sealing process [23].

Threat model of Intel SGX. An attacker is assumed to control the entire software stack of the underlying computing platform, including the hypervisor, BIOS, and OS. Specifically, within this context, the attacker can (1) execute arbitrary instructions at privileged levels; (2) launch, pause, and destroy enclave instances as desired; (3) control the software stack within the non-enclave environment, including tasks such as memory mapping, I/O device, and CPU scheduling [79].

TCB of Intel SGX. The TCB of SGX refers to all of a system’s hardware, firmware, and software components considered trusted and involved in providing a secure environment. A vulnerability in any TCB component would jeopardize the security of SGX. TCB size serves as a common metric to assess the security of a TEE design, where a smaller TCB size typically indicates fewer lines of code or hardware elements, implying a reduced vulnerability risk. However, comparing the security of two TEE designs based solely on TCB size is difficult and unreasonable [79]. Ongoing research

efforts are focused on building smaller TEE OS with memory-safe programming languages (e.g., Rust-SGX [142]).

2.3.2 Arm TZ Overview. TZ has been integrated into Arm processors (Cortex-A) since 2004 [6], primarily employed in mobile devices. Unlike Intel SGX, TZ is not inherently tailored for cloud platforms and lacks support for remote attestation, which would enable remote parties to verify the initial states of the TEE instances [79]. TZ operates around the notion of protection domains referred to as the **Secure World (SW)** and the **Normal World (NW)**. It also extends the concept of “privilege rings” by introducing **Exception Levels (ELs)** in the ARMv8 **Instruction Set Architecture (ISA)**. The “Secure Monitor” operates in the highest privilege level EL3, overseeing both the SW and NW. The untrusted OS runs within the NW, referred to as the **Rich Execution Environment (REE)**. Within the SW, the trusted OS kernel operates in EL1, while the secure userspace operates in EL0. The separation between SW and NW ensures that specific RAM regions and bus peripherals are designated as secure, limiting access solely to the SW.

Threat model of Arm TZ. Currently, there is a lack of a formal threat model for TZ. Commonly assumed attacker capabilities are as follows: The attacker (1) has full access to the REE of the device, encompassing the OS. This could be an authenticated (root) user, malicious third-party software installed on the device, or a compromised OS; (2) exploits unauthorized access to resources within the SW and extract sensitive information [114].

TCB of Arm TZ. Within the SW, the trusted OS operates in EL1 and provides runtime support to sustain the lifecycle of **Trusted Applications (TAs)**, ensuring their secure execution in user mode (EL0). The core of the trusted OS is the trusted kernel, which furnishes key OS primitives for the scheduling and management of TAs. The secure monitor implements mechanisms for secure context switching between worlds and operates with the highest privilege in EL3. Meanwhile, the TEE bootloader initiates the TEE system into a secure state [15]. In a nutshell, the trusted OS, secure monitor, and TEE bootloader constitute the TCB of a TZ-assisted TEE.

2.3.3 Virtualization-Based TEEs. Process-based TEEs such as Intel SGX require partitioning an application into a trusted part, the enclave, and an untrusted part, which can complicate development efforts as developers must adhere to this security model and segment their applications accordingly. In response, vendors now iterated on their previous trusted computing CPU extensions to facilitate virtualization-based confidential computing across various server platforms, eliminating the need for extensive modifications to application code. Here, we outline three such virtualization-based confidential computing solutions: Intel TDX [55], AMD SEV [5] and Arm CCA [7].

AMD SEV. AMD introduced SEV with the Zen architecture to enable confidential computing in cloud environments. Recent AMD SEV-**Secure Nested Paging (SNP)** [124] extends the capabilities of AMD SEV by introducing features such as secure nested paging to enhance the integrity of encrypted memory.

Arm CCA. Arm CCA is an advanced isolation technology introduced in Armv9. It preserves TZ’s NW and SW while introducing a new *realm* world designed for running multiple confidential “realms”. These realms, though isolated from other worlds, are managed by untrusted software components (e.g., hypervisor). A lightweight **Realm Management Monitor (RMM)** [8], operating within the hypervisor layer of the realm world, ensures memory isolation between realms. CCA incorporates a *root* world to accommodate the highest privilege monitor code stored in firmware [141].

Intel TDX. Intel TDX leverages Intel’s **Multi-Key Total Memory Encryption (MKTME)** [54] and a CPU-attested software module to facilitate the isolated execution of secure VMs. The TDX module, digitally signed and executed in a novel processor mode called SEAM, functions as a

separation kernel. It provides an interface to the hypervisor for scheduling, creating, and managing secure VMs, which are called *trusted domains*. For an in-depth comparison of AMD SEV, Arm CCA, and Intel TDX, we refer to the survey in Reference [45].

3 Overview and Taxonomy

Confidential computing on GPUs is an emerging field that ensures sensitive data processed on GPUs remains protected from unauthorized access and tampering even for attackers with privileged access. Other privacy-preserving techniques, such as secure multi-party computation, homomorphic encryption, and TEEs, offer different security guarantees. However, in this work, our primary focus is on the extension of TEEs to GPUs (i.e., GPU TEEs). Moving from homogeneous to such heterogeneous systems inevitably creates new threats and security vulnerabilities. It is crucial to understand these new attack vectors to inform the design of systems that can effectively mitigate them. In comparison to existing CPU TEEs, GPU TEEs present unique opportunities for attackers from several perspectives:

GPU software stack on CPU: GPU drivers handle critical aspects such as resource allocation, task scheduling, and dynamic power management. Specifically, in resource allocation, GPU drivers manage and allocate resources, including memory, processing units, and other hardware components. For task scheduling, they efficiently schedule and manage tasks to ensure optimal performance, balancing workloads across GPU cores. In dynamic power management, drivers adjust parameters such as voltage based on workload, optimizing performance and efficiency to extend battery life in mobile devices and reduce energy costs in data centers. In current GPU TEEs, the protection of GPU drivers, particularly their positioning in the CPU host, may introduce new attack surface. Furthermore, some state-of-the-art attacks against CPU TEEs may also affect GPU drivers.

Hardware components on GPU: GPU TEEs are still in the early stages of development. Most designs are experimental research and often lack extensive hardware modifications, resulting in insufficient hardware protection for components such as GPU memory, the PCIe bus, and power management components. Even products designed by GPU manufacturers (e.g., NVIDIA H100 GPU) might lack protection for certain hardware components, such as power management components. Thus, existing attacks against these components may also translate to weaknesses in GPU TEEs.

Architectural complexity: Although the security of GPUs is just starting to be explored, several vulnerabilities and attacks have already been documented. Due to the architectural complexity of GPUs, such as memory sharing among different concurrent kernels and the variety of GPU types (e.g., integrated and discrete), some existing GPU TEEs may not provide adequate protection. Additionally, many designs overlook cache-level access protection, making them susceptible to cache-based attacks on GPUs.

In Figure 3, we summarise our taxonomy underlying this survey based on relevant related work. In the remainder of this article, we first review and discuss existing GPU TEEs in Section 4, covering their threat models, hardware modifications, TCB size, and performance overhead. In Section 5, we give an overview of existing attacks targeting GPUs based on the attack types listed in Section 2. We also review state-of-the-art attacks against existing CPU TEEs, such as Intel SGX, Arm TZ, and AMD SEV in the supplemental material due to the limited space of this article. Finally, based on these GPU TEE designs and the summarized attacks, we discuss potential attacks against existing (and future) GPU TEEs in Section 6.

4 Confidential Computing on GPUs

We summarize the state-of-the-art GPU TEE designs based on Intel, AMD, and ARM in Table 2. Next, we first review all existing schemes and then discuss them based on aspects such as threat

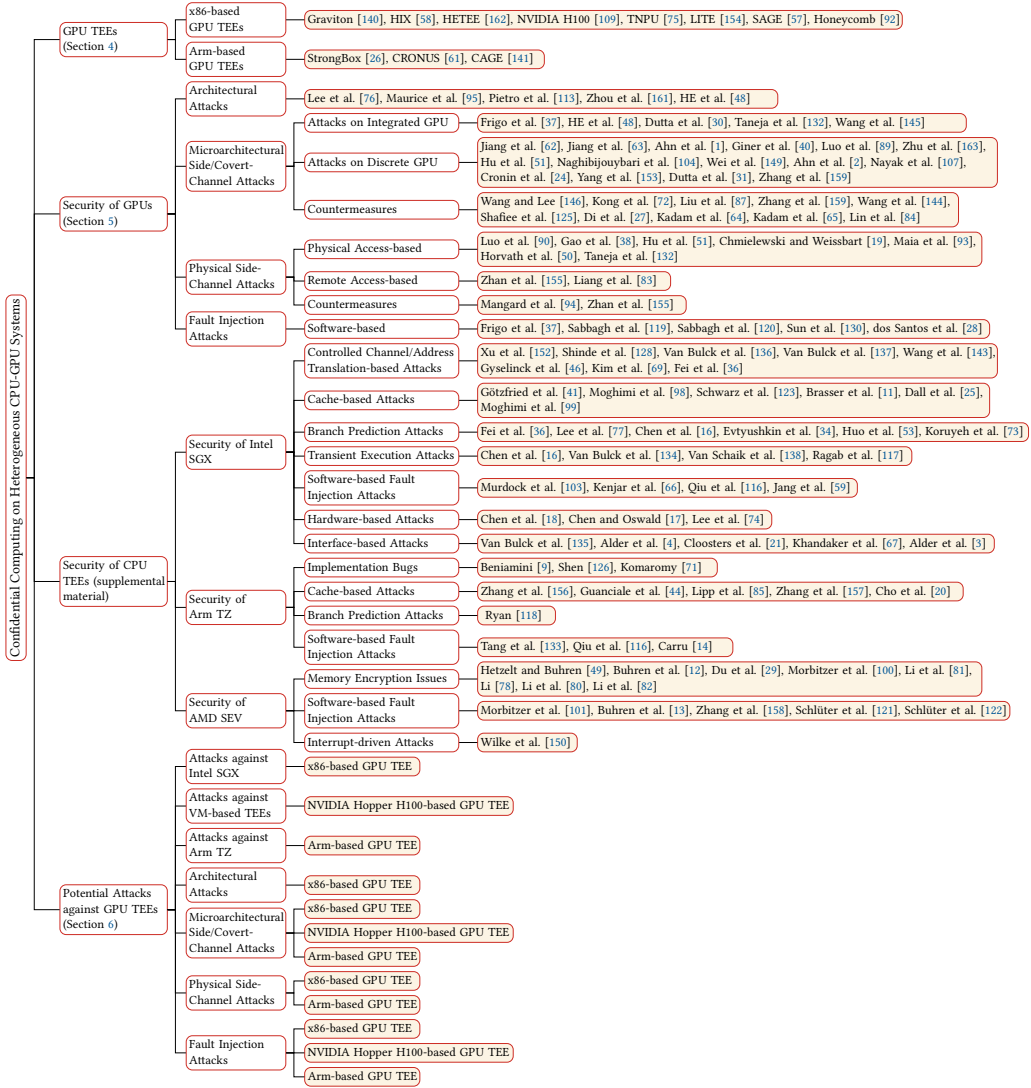


Fig. 3. Overview of confidential computing on heterogeneous CPU-GPU systems

models, hardware modifications, TCB size and performance overhead resulting from implementing trusted execution on accelerators.

4.1 Trusted Execution Environments on GPUs

Traditional CPUs and cloud systems have slowly adopted hardware-based TEEs to securely isolate computations from a malicious OS or certain hardware attacks, such as snooping on the memory bus. However, GPUs and their cloud deployments have largely yet to incorporate such support for hardware-based trusted computing. With large amounts of sensitive data being offloaded to GPUs for acceleration in cloud environments, especially for sensitive ML workloads, there is an urgent need to address the trusted computing requirements for ubiquitous GPUs. Researchers and commercial vendors have recently proposed various designs to support confidential computing on GPUs.

Table 2. Summary of Proposed Heterogeneous TEE for Accelerators

Scheme	ISA	Accelerator	Memory	GPU Driver Location	Methods	Key Components of GPU TEE				
						HW Changes	Attestation	Secure RM	Secure I/O	Integrity GPU ME
Graviton [140]	Intel x86	NVIDIA GPU	Dedicated	Untrusted OS	Modify GPU command processor	●	✓	✓	✓	✓
HIX [58]	Intel x86	NVIDIA GPU	Dedicated	Part inside CPU enclave	SGX Enclave	●	✓	✓	✓	✓
HETEE [162]	Intel x86	NVIDIA GPU	Dedicated	Inside CPU enclave	Dynamic allocation of accelerators within CPU enclave and extra FPGA	●	✓	✓	✓	✓
NVIDIA H100 [109]	Intel x86	NVIDIA GPU	Dedicated	Inside confidential VM	Hardware-level protection	●	✓	✓	✓	✓
TNPV [75]	Intel x86	NPU	Unified	Inside CPU enclave	SGX Enclave	●	✓	✓	✓	✓
LITE [154]	Intel/AMD x86	NVIDIA GPU	Dedicated	N/A	Software-based unified encryption domain	●	× [†]	× [†]	✓	✓
StrongBox [26]	Arm	Arm Mali GPU	Unified	Inside normal world	TZ	○	✓	✓	✓	✓
CRONUS [61]	Arm	General accelerators	Dedicated	Inside secure world	TZ virtualization	○	✓	✓	✓	× [†]
SAGE [57]	Intel x86	NVIDIA GPU	Dedicated	N/A	Software-based verification function	○	× [†]	× [†]	✓	✓
Honeycomb [92]	AMD x86	AMD GPU	Dedicated	Inside confidential VM	AMD SEV-SNP	○	✓	✓	✓	✓
CAGE [141]	Arm	Arm Mali GPU	Unified	Inside normal world	RME, GPC	○	× [†]	✓	✓	✓

Memory indicates whether the accelerators share the memory with the CPU or not. **HW Modification** explains if need modification on hardware (● = yes, ○ = no). **Secure RM** and **GPU ME** are *Secure Resource Management* and *GPU Memory Encryption*. † indicates that scheme needs additional support (e.g., hardware security features provided in NVIDIA H100 GPU or next-generation Arm devices), as they are not currently considering them.

x86-based GPU TEEs. Graviton [140] initiated the work of trusted execution on hardware accelerators like GPUs. Graviton directly establishes a TEE within the GPU, necessitating hardware modifications but avoiding the need for CPU-side TEE like SGX. Graviton establishes a secure channel between the untrusted host and GPU, monitoring command submission and data transfer by directing all resource allocation requests through the GPU's command processor instead of GPU driver. Nevertheless, for data protection, Graviton assumes the GPU's on-package memory can be trusted. The assumption is that even with physical access, it is extremely difficult for an attacker to breach the integrity of the GPU package and snoop on the silicon interconnect between GPU and stacked memory.

HIX [58] removes the key functionalities for controlling the GPU from the OS-resident driver and places those within a trusted *GPU enclave*, extending an SGX enclave and incorporating new SGX-style instructions like ECREATE and EADD. Furthermore, HIX establishes a secure hardware I/O path, preventing the OS from altering the virtual-to-physical address mapping for the GPU MMIO region and from deducing commands and data transmitted via MMIO and **Direct Memory Access (DMA)**. Like Graviton, HIX lacks a trusted GPU memory region, leaving data in GPU memory accessible in plaintext.

To eliminate the need for modifications to CPU or GPU chips, HETEE [162] introduces a data-center level GPU TEE designed to support extensive confidential computing, without any chip-level change. It facilitates dynamic allocation of computing resources for both secure and non-sensitive computing tasks across all servers within a rack. They incorporate a small TCB, such as a single security controller with customized code supporting task isolation, reducing design complexity and minimizing the attack surface associated with resource sharing with the untrusted OS.

Similar to HIX, Honeycomb [92] ensures secure communication between the user and the GPU by using a CPU-side TEE (e.g., AMD SEV-SNP) to transfer sensitive data to the GPU and manage access to GPU MMIO. Unlike previous designs, Honeycomb employs static analysis to confirm that mutually distrusting GPU applications are contained within their enclaves. This approach enables more efficient implementations by moving runtime checks to load time. Moreover, Honeycomb reduces the TCB by excluding both the user-space and kernel-space GPU drivers from it, instead using two security monitors to intercept and control all traffic between applications and the GPU. Honeycomb makes the same assumption as Graviton in trusting the GPU's device memory based on the fact that modern GPUs typically integrate device memory using 2.5D/3D silicon interposers within the same package.

SAGE [57] offers a software-based alternative for trusted GPU execution. It employs an SGX enclave running on the host as a local verifier and uses this enclave to bootstrap a software primitive

that establishes a dynamic root of trust on the GPU. It ensures that the user kernel on the untrusted device remains unmodified, is invoked for execution on the untrusted GPU, and runs without tampering. It complements existing hardware-based GPU TEEs.

Given that previous designs ignore memory security (i.e., they do not encrypt GPU memory), LITE [154] introduces a co-designed GPU TEE with CPU to establish a unified encryption domain, where data is stored in ciphertext form in GPU caches and memory. Specifically, LITE ensures that data enters the GPU chip encrypted. The software then decrypts the data using the **Arithmetic Logic Unit (ALU)**, operating on the data in registers. Additionally, data may be temporarily stored in plaintext in the on-chip shared memory.

TNPU [75] proposes tree-less off-chip memory protection for **Deep Neural Networks (DNN)** accelerators. The core idea is to group contiguous memory blocks into tiles and provide freshness guarantees at the tile level. TNPU employs a dedicated software module running on the host CPU to manage version numbers, which are stored in a “Tensor Table” within the host CPU’s secure memory. The tensor table is protected by an integrity tree.

NVIDIA’s H100 Tensor Core GPU [109], based on the Hopper architecture, introduces advanced features for confidential computing. To enable confidential computing with the H100, it requires specific CPU TEEs, such as Intel TDX, AMD SEV-SNP, and Arm CCA. In addition to ensuring the confidentiality and integrity of data and code, H100 can defend against basic physical attacks targeting the PCIe bus and **Double Data Rate RAM (DDR)**. H100 supports three different operation modes, namely *CC-Off*, *CC-On*, and *CC-DevTools*. In *CC-On* mode, the H100, along with the drivers on the CPU, fully activates all available confidential computing features. Some hardware resources, such as performance counters, are disabled in *CC-On* mode to prevent potential side-channel attacks, as performance counters could be used to infer the behavior of device usage [106]. However, details about specific hardware changes, such as memory encryption in the H100, remain unclear at present.

Arm-based GPU TEEs. Previous designs for Intel/AMD-based platforms cannot be readily extended to Arm GPUs due to numerous architectural differences. StrongBox [26] thus introduces the first GPU TEE for secure general computation on Arm endpoints without necessitating hardware modifications or architectural changes. It deploys the StrongBox runtime within the secure monitor to ensure that the GPU executes in isolation. Through a combination of Stage-2 translation and the **TrustZone Address Space Controller (TZASC)**, StrongBox specifies six distinct access permission types to effectively manage secure access from DMA, GPU, and other peripherals.

CRONUS [61] introduces a design that supports various heterogeneous accelerators, enables spatial sharing within a single accelerator, and ensures robust isolation across multiple accelerators. Leveraging Arm secure virtualization technology, CRONUS partitions heterogeneous computation into isolated TEE enclaves. Each enclave encapsulates a specific type of computation like GPU computation and allows multiple enclaves to spatially share a single accelerator.

To support confidential GPU computing on next-generation Arm devices, CAGE [141] presents a new scheme for Arm CCA’s realm-style architecture. It offloads resource-intensive but data-independent tasks to NW, while protecting sensitive data within realms through a shadow task mechanism. By configuring **Granule Protection Check (GPC)**, CAGE restricts access from untrusted components. Additionally, it isolates realms in GPU computing by providing each realm with a distinct GPU memory view in the GPU GPC. CAGE retains high hardware compatibility, as it requires no additional hardware or modifications to existing CPU and GPU configurations.

4.2 Discussion

In this section, we discuss several key metrics in designing an effective and secure GPU TEEs.

Comparison to CPU TEEs. While both CPU and GPU TEEs aim at providing secure and isolated execution environments, their designs differ significantly due to fundamental differences in

microarchitecture and usage patterns. CPU TEEs, such as Intel SGX and AMD SEV-SNP, operate in control-centric environments with centralized execution and predictable memory behavior, making them well-suited for fine-grained access control, secure paging, and privilege enforcement. In contrast, GPUs are optimized for massive data-parallel workloads, with thousands of lightweight threads, wide memory bandwidth, and deep memory hierarchies. These characteristics make it difficult to directly apply traditional TEE techniques like enclave management or synchronized memory encryption. Moreover, GPUs often lack built-in privilege separation and rely on the CPU for task scheduling and memory control, expanding the attack surface. As a result, GPU TEE designs typically require either hardware modifications to decouple from the host (e.g., Graviton, HIX) or leverage CPU-side TEEs for protection and communication (e.g., Honeycomb, SAGE). This architectural gap explains the greater diversity and complexity in GPU TEE designs compared with their CPU counterparts.

Threat models. Essentially all GPU TEE designs adopt a threat model similar to that of CPU TEEs, aiming to create secure execution environments for GPUs. Specifically, they consider an adversary capable of controlling the entire software stack, including device drivers, the guest OS, and the hypervisor. Additionally, the adversary is assumed to have limited physical access to the hardware, allowing for passive physical attacks, such as snooping on PCIe traffic. This means that such an adversary could potentially leak or tamper with sensitive data and execution results of GPU applications. They could also access or tamper with user data in DMA buffers or with commands submitted by the victim application to the GPU.

Regarding GPU memory protection, many designs assume that the adversary cannot access secrets in the GPU memory or explicitly state that physical attacks targeting memory are out of their scope. For instance, Graviton [140] and Honeycomb [92] trust the device memory of the GPU, as modern GPUs typically integrate device memory using 2.5D/3D silicon interposers within the same package. These designs mainly focus on preventing adversaries from accessing sensitive data stored in GPU memory with finer-grained access control. To fill this gap, LITE [154] designs a software-based unified encryption domain to protect sensitive data stored in GPU memory, including L1/L2 cache and device memory. Additionally, NVIDIA H100 [109] and TNPU [75] support hardware-based memory encryption, while CAGE [141] requires hardware-assisted memory encryption support built into future CCA devices.

Hardware changes. Earlier works like Graviton [140] and HIX [58] require non-trivial modifications to peripheral components, such as the GPU's command processor, the introduction of new SGX instructions, specific **Memory Management Unit (MMU)** page table walkers, and other hardware changes. The extent to which a GPU TEE design modifies hardware components determines its practical compatibility, i.e., its suitability for various GPUs. Many designs necessitate hardware changes that have long lead times before they can be deployed in production environments. Consequently, works like LITE [154], SAGE [57], and Honeycomb [92], seek purely software-based approaches for trusted GPU execution. CAGE [141] primarily leverages generic hardware features in Arm CCA and GPUs without requiring additional hardware changes or customization of existing CPUs and GPUs.

TCB size. One of the key challenges in GPU TEE design is reducing the TCB size as much as possible. A large software stack increases the TCB, which can introduce more vulnerabilities and hence threaten security. An effective and efficient way to reduce the TCB is to place most of the GPU software stack in the untrusted OS, while keeping only the key functions for controlling the GPU in a trusted area of the CPU, such as an SGX enclave. For example, CAGE [141] achieves a 'thin' TCB for realms by using its shadow task mechanism instead of loading the heavyweight GPU

software into the TEE. It introduces 2--26K lines of code for a CPU-side isolation and 1,301 lines of code for monitor, resulting in the lowest TCB size among current Arm-based GPU TEEs.

Performance overhead. The ultimate goal of designing a GPU TEE is to ensure strong isolation of GPU execution while minimizing the performance overhead associated with providing security guarantees. Comparing the performance overhead of different schemes is challenging due to the costly re-implementation efforts required, as each scheme targets different platforms and is implemented under varying conditions. For example, in HIX, the software components are implemented on top of an emulated system (like KVM and QEMU), which supports hardware modifications. Several factors, such as the number of cryptographic operations, the frequency of switching between secure and normal worlds, and the integrity checks, play significant roles in optimizing performance compared with insecure baselines on GPUs. For example, CAGE [141] outperforms StrongBox [26] by 1.96%–13.16% on six Rodinia benchmarks. This performance gain is attributed to CAGE's ability to securely transfer plaintext data to buffers and store the results without introducing additional cryptographic operations. It is reasonable to speculate that the NVIDIA H100 GPU introduces a smaller performance burden due to its hardware-based encryption support. It would be insightful to conduct a comprehensive performance comparison across different platforms once the entire workflow for confidential computing on the NVIDIA H100 GPU is fully available in the future.

Tradeoff between cryptographic methods and GPU TEEs. In general, while cryptographic approaches like HE and MPC offer strong confidentiality guarantees, they remain impractical for many GPU workloads due to significant computation/communication overhead, memory usage, and developer complexity. Recent GPU-based MPC systems such as CryptGPU [131], Piranha [148], and Orca [60] show that performance is highly sensitive to the number of computing participants and choice of operators, making them difficult to develop. Piranha further highlights that communication overhead becomes a major bottleneck in wide-area network settings—an important consideration since real-world deployments often involve geographically distributed GPU providers. For HE, even with dedicated HE accelerator support, as in TensorFHE [35], significant modifications are needed to underlying schemes (e.g., CKKS) and operations (e.g., NTT). Applying HE to ML workloads also requires careful tuning of HE parameters and may necessitate changes to model architecture, complicating deployment in applications. Therefore, GPU TEEs offer a more practical path forward, enabling protection of existing workloads with less developer effort and minimal modification to ML models. While GPU TEEs are still in their early stages and only applicable on certain machines, we believe they hold strong potential for future GPU-based applications, especially in ML.

5 Security of GPUs

Due to their widespread adoption as accelerators, GPUs have come under scrutiny from both offensive and defensive perspectives in recent years. Researchers have explored GPU vulnerabilities at hardware architecture and software levels, showcasing practical attacks and suggesting mitigations. We believe that summarizing and analyzing attacks on GPUs will facilitate the exploration of potential attacks on GPU TEEs. While Naghibijouybari et al. [105] offer a comprehensive survey categorizing GPU security vulnerabilities and potential countermeasures, their analysis does not cover significant advancements that have appeared since the latter half of 2022. Moreover, their emphasis primarily is on microarchitectural attacks across diverse accelerators without considering attacks that leverage physical access or measurements. Nonetheless, the consideration of physical channels is essential for potential attacks on GPU TEE.

In this section, we review existing GPU attacks, including microarchitectural side-channel/covert-channel attacks, physical side-channel attacks, and fault injection attacks. Table 3 provides a

Table 3. Comparison of Different Attacks on Heterogeneous Systems (Oldest Results First)

Work&Year	Accelerator	Threat Model		Physical	Attack Scenarios & Goals	Method	Main results
		Model	Permissions				
Architectural Attacks							
Lee et al.'14 [76]	NVIDIA/AMD GPU	sGPU	User-level permissions	×	Infer which web pages a victim user has visited	Exploit that GPUs do not initialize newly allocated memory pages	95.4% accuracy
Maurice et al.'14 [95]	NVIDIA GPU	sGPU	Require root privileges based on accessing GPU memory via GPU runtime or PCI configuration	×	Recover data of victim's previously execution on GPU in virtualized environment	Exploit the vulnerability of memory clearing on GPU	Access data stored in GPU global memory after soft reboot
Pietro et al.'16 [113]	NVIDIA GPU	sGPU	User-level permissions	×	Read information stored in memory and infer the AES keys	Leverage the lack of adequate data allocation and data cleaning	70%, 28% leakages
Zhou et al.'16 [161]	NVIDIA/AMD GPU	sGPU	User-level permissions	×	Recover users' profile images and email contents	Reconstruct images from identified image-like tiles in GPU memory	79.3% successful recovery
Microarchitectural Side-Channel/Covert-Channel Attacks							
Luo et al.'15 [90]	NVIDIA GPU	sGPU	Unprivileged attacker has physical access to the victim GPU	✓	Recover all of the last round AES key byte	Apply correlation power analysis to analyze the power consumption of the GPU	Recover the key with 160,000 power traces
Jiang et al.'16 [62]	NVIDIA GPU	sGPU	Send plaintext to victim GPU for encryption with user-level permission	×	Recover the 16-byte last-round key of the AES	Leverage the relation between encryption kernel's runtime and the key	30 min with 1M timing samples
Jiang et al.'17 [63]	NVIDIA GPU	sGPU	Send plaintext to victim GPU for encryption with user-level permission	×	Recover the 16-byte last-round key of the AES	Exploit timing variability due to shared memory bank conflicts.	Use 10M timing samples
Naghijouybari et al.'17 [104]	NVIDIA GPU	sGPU	Trojan and spy co-exist on a GPU, both with user-level permission	×	Infer transmitted data	Reverse engineer warp schedulers; learn contention on caches and memory	4 MB/s (error-free)
Gao et al.'18 [38]	NVIDIA GPU	sGPU	Unprivileged attacker has physical access to the victim GPU	✓	Recover AES private key	Sample MMIO traces via trigger; exploit parallel MMIO leakages	Recover the key with 11k MMIO traces
Naghijouybari et al.'18 [106]	NVIDIA GPU	sGPU	Spy co-locates with the victim program with user-level permission	×	Infer victim's accessed website data and the number of neurons	Learn contention via memory allocation, performance counter, and timing	90%, 94.8%, 88.6% accuracy
Luo et al.'19 [89]	NVIDIA GPU	sGPU	User-level permissions; available to timing information	×	Recover RSA private key	Capture the parallel characteristics of an RSA on GPU via timing model	Recover 512/2048-bit key in 15 min/8 h
Wei et al.'20 [149]	NVIDIA GPU	sGPU	Spy co-locates with the victim program with user-level permission	×	Recover the layer composition and hyper-parameters of DNN	Leverage context-switching penalties when switching different kernels	76.9%-100% accuracy
Hu et al.'20 [51]	NVIDIA GPU	sGPU	Physically access the victim GPU with user-level permissions	✓	Obtain the complete model architecture information	Learn the inter-layer temporal association information from kernel execution	75.9% accuracy
HE et al.'20 [48]	Intel HD 530	iGPU	Unprivileged adversary and the victim co-locate on the same GPU, but are isolated	×	(1) Recover AES key; (2) monitor the browser activity; (3) conduct a covert channel to exchange data	Exploit context management with unclear registers and shared memory during context switches	Recover AES-128/192 in 0.15 s/2 min; 90.7% accuracy; 1.3–8 Gbps
Ahn et al.'21 [2]	NVIDIA GPU	sGPU	Trojan and spy co-exist on a GPU, both with user-level permissions	×	Infer transmitted data	Exploit interconnect channel contention	24 MB/s
Cronin et al.'21 [24]	Apple GPU	sGPU	Exploit JavaScript website and trick user into installing a malicious app	×	Track the user's web activity	Exploit the shared cache between the CPU-cores and peripherals	90%, 95% accuracy
Zhu et al.'21 [163]	NVIDIA GPU	sGPU	Install a PCIe bus snooping device to monitor and log the PCIe traffic	×	Recover the DNN model (semantically identical architecture)	Reverse engineer GPU command headers; collect PCIe packets via snooping	Similar accuracy as original ones
Ahn et al.'21 [1]	NVIDIA GPU	sGPU	Send plaintext to victim GPU for encryption with user-level permission	×	Recover the 16-byte last-round key of the AES	Recover early/late key bytes via timing correlation and cache-collision	Use 100k timing samples
Dutta et al.'21 [30]	Intel HD Graphics	iGPU	Spy and trojan run on CPU and GPU with user-level permissions	×	Infer transmitted data and track user's cache activity	CovA1: prime+probe attack on LLC; CovA2: observe delays on the ring bus	CovA1: 120 KB/s, CovA2: 400 KB/s
Nayak et al.'21 [107]	NVIDIA GPU	sGPU	User-level permissions	×	Infer the accessed data from Virginian database	Leverage the shared last-level TLB (reverse engineering) and prime+probe	81 KB/s (MPS-enabled)
Yang et al.'22 [153]	Qualcomm Adreno	sGPU	Install and launch an eavesdropping application on the victim device	×	Eavesdrop the user's input credentials (login usernames and passwords) through onscreen keyboard	Read raw values from the GPU device file and identify GPU events from key presses via ML models	Correctly infer more than 80% of user's credential inputs
Zhan et al.'22 [155]	NVIDIA/AMD GPU	sGPU	Unprivileged attacker eavesdrops on a victim	✓	Website fingerprinting and keystroke timing inference	Exploit EM caused by DVFS	95.6% accuracy and 90% keystrokes
Liang et al.'22 [83]	NVIDIA GPU	sGPU	Attacker and victim GPUs may be separated by meters	✓	Infer the number of layers/kernels, sizes of layers and strides	Exploit far-field EM signals emanated from a GPU	Recover VGG's and ResNet's parameters
Maia et al.'22 [93]	NVIDIA GPU	sGPU	Unprivileged attacker with physical proximity	✓	Infer NN architectures and layer parameters	Exploit the magnetic flux emanating from a GPU's power cable	Up to 96.8% accuracy
Horvath et al.'23 [50]	NVIDIA GPU	sGPU	User-level permission with physical access and know the CNN architecture	✓	Recover the weights and biases of CNN	Correlation electromagnetic analysis	N/A
Dutta et al.'23 [31]	NVIDIA GPU	mGPU	Spy and victim connect via NVLink with user-level permissions	×	CovA: infer message; SCA: infer the number of neurons of NN	Mount Prime+Probe attack on the L2 cache of victim GPU	CovA: 4 MB/s; SCA: 99.91% accuracy
Zhang et al.'23 [159]	NVIDIA GPU	mGPU	Unprivileged spy and trojan on an MIG-enabled GPU	×	Infer transmitted data and classify the running ML frameworks	Mount Prime+Probe attack on the shared L3-TLB	31.47 KB/s
Taneja et al.'23 [132]	Various GPUs	iGPU, sGPU	Read frequency, power, and temperature from internal sensors (may require privileged access)	✓	(1) Observe CPU/GPU behaviour; (2) recover images and user's browsing history; (3) fingerprint websites	(1) Use data from internal sensors; (2) observe throttling via timing; (3) leverage frequency and power bursts on iGPUs	Pixel stealing with accuracy of 60%–90%
Wang et al.'24 [145]	Various GPUs	iGPU, sGPU	Observe coarse-grained pattern redundancy using timer	×	Perform cross-origin pixel stealing attacks in the browser	Infer data-dependent DRAM traffic and cache usage via compression channel	Fixed stealing with accuracy 68.6%–99.6%
Giner et al.'24 [40]	NVIDIA/AMD GPU	sGPU	SCA: Victim passively visits attacker's webpage; CovA: Spy accesses the website visited by victim	×	SCA: Detect typed characters and recover the victim's AES key; CovA: Infer transmitted data	Exploit cache contention on the L2 cache via Prime+Probe attack, parallelize eviction set construction	SCA: 98% accuracy, recover key in 6 min; CovA: 10.9 KB/s
Fault Injection Attacks							
Frigo et al.'18 [37]	Intel HD, Qualcomm Adreno	iGPU	Access iGPU through a malicious application or when victim visits malicious JavaScript website	×	Escape the Firefox sandbox on Android platforms and break ASLR	Detect contiguous memory allocation via SCA and then trigger Rowhammer bit flips	Hijack the browser and break ASLR in 116 s and 27 s
Sabbagh et al.'20 [119]	AMD GPU	sGPU	DVFS interfaces access	×	Recover AES key	Exploit DVFS to introduce random faults during kernel execution	Recover key in 15 min
dos Santos et al.'21 [28]	NVIDIA GPU	sGPU	Inject faults into pipeline registers, ALU, and memory	×	Improve the probability of incorrect output by injecting faults	Combine register transfer level FIA with software FIA	Underestimate the error rate by up to 48%

summary of attacks, target GPUs, threat model, attack goals and methods. Based on Table 3, we highlight several interesting take-away points: ❶ Most attacks target single discrete GPU systems, with memory and timing behavior serving as the primary leakage vectors. However, recent research has begun to explore attacks on multi-GPU and multi-instance GPU environments, revealing new vulnerabilities in GPU-to-GPU communication and resource partitioning. ❷ A majority of attacks can be carried out with only user-level permissions, indicating that isolation mechanisms in modern GPU systems remain inadequate, particularly in multi-tenant or cloud-based deployments. ❸ Attacks span across multiple GPU vendors (e.g., NVIDIA, AMD, Qualcomm) and consistently aim at recovering sensitive information, such as AES keys, **Neural Networks (NNs)** model parameters,

and user input keystrokes. Notably, attacks targeting more complex GPU workloads, such as **Large Language Models (LLM)**, remain largely unexplored. ❹ While impactful, fault injection and covert channels appear less frequently in the literature. These often require stronger attacker capabilities or intricate coordination between software and hardware layers, which may explain their limited exploration. ❺ Emerging attacks have begun to explore cross-GPU timing channels and remote leakage via browsers, suggesting that cloud-hosted GPU services—especially those accessible by multiple users—may become an increasingly important and vulnerable attack surface.

5.1 Architectural Attacks

There has been an increase in research examining attacks that target GPUs or their software stack on the CPU. Schemes [76, 95, 113, 161] focus on information leakage due to leftover memory from processes that recently terminated. Lee et al. [76] exploit the vulnerability arising from the GPUs not initializing newly allocated memory pages, thereby revealing the victim's data stored in GPU memory both during its execution and immediately after termination. In practice, they infer the webpages visited by a victim user when analyzing remaining textures from Chromium and Firefox. Maurice et al. [95] recover data from previously executed GPU applications by observing that GPU global memory is not always zeroed. Pietro et al. [113] leverage interleaved access and inadequate data handling to read information stored in shared or global memory by another process. Zhou et al. [161] demonstrate how malicious programs can exploit GPU memory management strategies to breach the memory isolation boundary and recovers images from other processes. HE et al. [48] observed a vulnerability in Intel integrated GPU caused by flawed context management, where residual register values and shared memory are not cleared during a context switch. This class of vulnerability can usually be fixed by clearing memory (i.e., memory zeroing) before reallocating it to another application.

Takeaway points. Architectural attacks often exploit leftover data in GPU memory due to missing or incomplete memory clearing. These attacks require only user-level access and affect both integrated and discrete GPUs, showing that such flaws are common across vendors. While many of these vulnerabilities have been patched in recent years, they highlight past weaknesses in GPU memory isolation.

5.2 Microarchitectural Side-Channel/Covert-Channel Attacks

In this subsection, we review existing attacks on GPUs w.r.t. microarchitectural side-channel and covert-channel attacks.

5.2.1 Attacks on Integrated GPUs. Attacks on integrated GPUs are gaining attention due to their security risks. The integration opens up new attack opportunities by exploiting shared resources, leading to cross-component microarchitectural attacks. A key requirement for these attacks is co-location, where the attacker runs code close enough to interact with the victim [105]. This interaction could involve inducing faults or contention to measure side-channel leakage. Therefore, integrated GPUs, which share the same chip as the CPU, offer more attack opportunities.

Dutta et al. [30] develop two covert channels, enabling two malicious applications located on separate components (CPU and iGPU) to exchange confidential data through shared hardware resources. One channel exploits the shared **Last-Level Cache (LLC)** within Intel's integrated GPU, while the other relies on the contention from the ring bus. Wang et al. [145] develop cross-origin pixel stealing attacks in web browsers by exploiting the iGPU compression channel to induce data-dependent DRAM traffic and cache utilization. Given that the LLC is shared between the CPU and GPU, they demonstrate how to use LLC walk time to infer the GPU-induced CPU

cache state changes. Wang et al. [147] extend the previous timing side-channel attack on AES implementation [62] to the iGPU.

Some recent studies have also delved into GPUs such as the Qualcomm Adreno GPU and Apple GPU. Building upon Arm's system-level cache, Cronin et al. [24] exploit the GPU and shared cache architecture of Arm DynamIQ to create a website fingerprinting side channel. Yang et al. [153] target the Qualcomm Adreno GPUs, investigating the GPU overdraw induced by rendering popups of user key presses. Each key press leads to unique variations in selected GPU performance counters, allowing accurate inference of these key presses. To access GPU performance counters on Android, they read the raw values of GPU PCs from the GPU device file (e.g., Qualcomm Adreno GPU drivers' open-source header file, `msm_kgsl.h`).

Takeaway points. Integrated GPUs introduce new security challenges due to their tight coupling with CPUs and shared microarchitectural resources, such as LLC and interconnects. This shared environment enables cross-component attacks, exploiting shared caches, compression channels, and GPU performance counters. These attacks are particularly concerning in web and mobile environments, where integrated GPUs are widely deployed and users have minimal control over low-level hardware access.

5.2.2 Attacks on Discrete GPUs. Attacks on discrete GPUs can generally be categorized in two ways: (i) where the spy and victim (or trojan) applications are co-located on a single GPU, denoted as *sGPU*; and (ii) where the spy and victim (or trojan) applications are co-located on different GPUs, denoted as *mGPU*. For instance, this scenario could involve a system equipped with multiple GPUs, such as NVIDIA's Pascal-based DGX-1 system comprising eight Tesla P100 GPUs. Alternatively, it could involve a system equipped with an MIG-enabled GPU, where a single physical GPU is partitioned into multiple instances, each acting as an independent GPU with dedicated resources, such as NVIDIA A100 and H100.

Attacks on a single GPU. This category of attacks typically assumes that an attacker with only user-level permissions either launches a GPU kernel on a single GPU to gain insights from measurements such as execution time, performance counters, and other leakage vectors; or co-locates with the victim process on a single GPU to establish a covert channel. In the following, we review the existing research and categorize them into the following aspects:

Recovering private encryption keys in GPU-based cryptography implementations: Recent works demonstrate the capability of recovering encryption keys (e.g., AES and RSA private keys) from encryption applications running on GPUs. Jiang et al. [62] exploit timing differences between addresses generated by different threads accessing memory, where execution time is affected by the number of unique memory requests after coalescing. Consequently, the key affects the address pattern accessed by threads, thereby impacting the observed runtime of the AES encryption. In their another study [63], they establish a correlation between the execution time of a warp's table lookup and the number of shared memory bank conflicts generated by threads within the warp. These key-dependent timing differences are then used to correlate the measured execution time with the key at the last round of the AES encryption as it executes on the GPU. Ahn et al. [1] exploits negative timing correlation to recover earlier key bytes of AES, while employing cache-collision attacks for the latter AES key bytes. More recently, Giner et al. [40] recover keys from an AES T-table GPU implementation. They allocate a sizable buffer to occupy a significant portion of the cache, execute an AES encryption, and then use Prime+Probe to identify evicted buffer offsets that correlate with the encryption's inputs and outputs. They demonstrate superior performance by recovering the entire AES key within 6 min, whereas previous studies took between 15 and 30 min. Furthermore, aside from GPU-based AES implementation, Luo et al. [89] present a timing

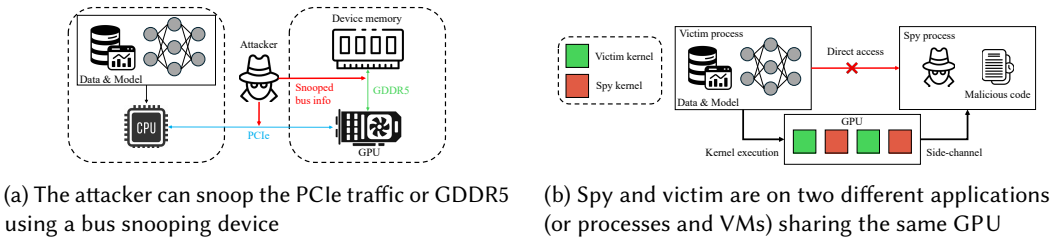


Fig. 4. Attacker models for inferring NN models on a single GPU.

side-channel attack on an RSA implementation. They use the correlation between decryption time and the number of reductions per decryption window to extract the RSA private key.

Inferring sensitive information from GPU-based NN: Other attacks exploit side-channel leakage to infer information from ML models like NN. Zhu et al. [163] and Hu et al. [51] focus on the attack surface of the bus, particularly PCIe (as depicted in Figure 4(a)). Zhu et al. [163] reverse-engineer the critical data structures (e.g., GPU command headers) offline and collect victim PCIe packets via a snooping device to reconstruct models in the online phase. The stolen DNN models have the same parameters and semantically identical architecture as the original ones. One potential mitigation for this attack is encrypting PCIe traffic. In contrast, Hu et al. [51] assume that the adversary cannot access data passing through buses, only the addresses, allowing attacks even with encrypted data. They conduct bus snooping attacks on discrete GPU's memory and PCIe bus, obtaining kernel read/write access volume and memory address traces. Correlating extracted architectural events with model internal architectures. Another class of attacks occurs between two concurrent GPU applications, as shown in Figure 4(b). In [104], they (i) implement website fingerprinting attack via the GPU memory utilization API or GPU performance counters; (ii) monitor user's web activities during interactions or while typing characters on a keyboard through keystroke timing analysis; (iii) utilize a CUDA spy within the computational stack of GPU to deduce the NN structure by leveraging shared resources. In contrast to [104], Wei et al. [149] consider fine-grained time-sliced sharing of applications on different VMs and exploit context-switching penalties on performance counters as leakage vector to recover the layer composition and hyperparameters of a DNN.

Most of these attacks require extensive reverse-engineering of NVIDIA GPU details. This is because GPU driver and internal GPU workings are closed-source, and crucial data structures are left undocumented (cf. Section 2.1). In other words, these factors make the attack more difficult to some extent, but do not fundamentally prevent them.

Covert-channel attacks where both spy and victim concurrently run on a single GPU: As summarized in Figure 5, the key point of a successful covert-channel attack is identifying the shared resources utilized by both the trojan and spy, often requiring extensive reverse engineering of GPU architectures. This process is necessary to uncover details about GPU components, such as the TLB and other shared resources, which can be exploited for covert communication. Naghibijouybari et al. [104] first reverse-engineer the hardware block and warp-to-warp schedulers to establish the co-location, then learn contention on caches, functional units and memory, and finally construct covert channels based on these resources with an error-free bandwidth exceeding 4 Mbps. With this, they recover the number of neurons in an NN. Ahn et al. [2] initially reverse-engineer the structure of on-chip networks and find the hierarchical organization of the GPU leads to the sharing of interconnect bandwidth among adjacent cores. They then exploit the contention of the interconnect channel, leverage clock registers for sender-receiver synchronization and ultimately establish microarchitectural covert channels with a high bandwidth of 24 Mbps. Nayak et al. [107] reverse-engineer the details of the TLB hierarchy and the details of all TLB levels, such as the

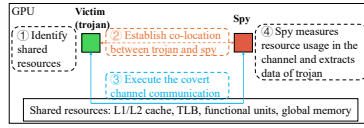


Fig. 5. Key steps of covert-channel attacks on a single GPU.

number of entries and associativity, of the NVIDIA 1080Ti. They create a covert timing channel via the shared L3 TLB using Prime+Probe. With MPS, they increase the bandwidth of the GPU's L3 TLB-based covert channel to 81 Kbps. Unlike previous work relies on access to the native GPU APIs through CUDA or OpenGL for high-precision performance counters monitoring, Giner et al. [40] do not rely on WebGPU providing hardware timers. Instead, they use a shared memory buffer and a dedicated thread to increment a shared variable constantly, functioning as a timer. With this timer and their GPU-accelerated cache eviction set construction, they establish a Prime+Probe cache covert channel for the L2 cache with a bandwidth of 10.9 KB/s.

Attacks on multiple GPUs. In NVIDIA's DGX machines, where discrete GPUs are connected via NVLink and PCIe, Dutta et al. [31] reverse-engineer the cache hierarchy and show that an attacker on one GPU can create contention in the L2 cache of another GPU through NVLink. They establish a Prime+Probe covert-channel across two GPUs, achieving a bandwidth of 4 MB/s. Additionally, their side-channel attack allows an attacker to monitor another application running on different GPUs, enabling the recovery of the number of neurons of the NN. By reverse-engineering the TLB structure in recent NVIDIA GPUs such as A100 and H100, Zhang et al. [159] discover a design flaw in the NVIDIA MIG feature: MIG does not partition the last-level TLB, which is shared by all GPU instances. They dump GPU memory to locate corresponding table entries and access the target entry through the 1 MB GPU memory window (MMIO) for runtime modifications. They finally establish a cross-GPU-instance covert-channel based on the shared TLB. In summary, both studies begin by identifying shared resources present in multi-GPU systems, such as the shared remote L2 cache [31] and the shared L3 TLB [159], and then create visible contention on these shared resources.

Takeaway points. ❶ Reverse engineering remains key to GPU attacks, highlighting both the limitations of security through obscurity and the lack of transparency from vendors that impedes systematic evaluation. ❷ Attack bandwidths have significantly improved and are now comparable to early CPU-based covert channels, reflecting the growing sophistication of GPU exploitation. ❸ Isolation mechanisms such as MIG may have flaws. Logical partitioning alone does not guarantee security and requires careful validation. ❹ Attacks leveraging NVLink and PCIe show that GPU interconnects are an emerging and under-protected attack surface, particularly in AI datacenter environments. ❺ ML workloads, especially DNNs, remain prime attack targets. Yet, the security of newer models such as LLMs and diffusion models across multi-GPU systems remains largely unexplored.

5.2.3 Countermeasures. The countermeasures can be summarized as follows:

(i) *Partitioning resources:* Co-location between the attacker and the victim is essential for those attacks that exploit shared microarchitectural resources. One solution to mitigate this threat is to partition resources (e.g., LLC) either statically or dynamically [72, 87, 146]. By assigning the spy and trojan to different cache partitions, they cannot replace each other's cache lines. The MIG introduced in Section 2.1 potentially mitigates these attacks by isolating resources between independent GPU instances. However, Zhang et al. [159] point out that the last-level TLB between MIG-enabled GPU instances is not properly partitioned, allowing for the creation of a covert channel using this vulnerability.

(ii) *Eliminating contention*: Contentions among processes can be eliminated by controlling traffic in memory controllers. This involves grouping memory requests from each processor into the same queue, potentially accessing the same memory bank/port [125, 144]. For example, TLB-pilot [27] binds thread blocks from different kernels to different groups of streaming multiprocessors by considering hardware isolation of last-level TLB and the application's resource needs. It gathers kernel information before kernel launches and reduces load imbalance by coordinating software and hardware-based scheduling and employing a kernel splitting strategy. There are also defences against coalescing-based correlation timing attacks. For example, Kadam et al. [64, 65] add extra memory accesses to make execution time less predictable. To protect against timing and cache-based attacks in GPU-based AES, Lin et al. [84] use scatter and gather techniques to reorganize AES tables, avoiding key-dependent lookups from timing or address leakage.

5.3 Physical Side-Channel Attacks

We classify attacks allowing an attacker to have direct proximity or contact with the GPU or its surrounding hardware components as *physical access-based* attacks. Such an attacker is physically close to the device and can directly interact with it. This may involve placing sensors or monitoring equipment directly on or near the device to capture physical signals or emissions. Note that we term attacks that perform monitoring or manipulation of the physical properties of the target device from a distance, without direct physical access to the device itself, as *remote access-based* attacks (called hybrid attacks in Reference [132]). Such attacks use e.g., wireless sensors or network-connected devices to capture signals or emissions emitted by the target device from a distance.

5.3.1 Physical Access-Based. Luo et al. [90] and Gao et al. [38] focus on recovering the AES private key by exploiting physical properties. Luo et al. utilize correlation power analysis to analyze the GPU's power consumption (i.e., power traces), necessitating the attacker to instrument the power supply. Gao et al. sample EM traces employing a precise trigger mechanism and establish a heuristic leakage model to exploit the concurrent MMIO leakages in parallel scenarios. This method requires removing the GPU's heat sink and positioning a probe near the GPU chip surface.

Hu et al. [51] employ MMIO side-channel attacks to capture memory access volume and kernel execution time by targeting CPU-GPU interconnections or co-locating a CUDA spy. This allows them to deduce the model architecture without any prior knowledge of the NN model. Similarly, Chmielewski and Weissbart [19] use simple MMIO analysis to recover partial information of an NN model. Maia et al. [93] show how to recover the full model architectures of 64 NNs by exploiting the magnetic flux emanating from a GPU's power cable via a physical sensor. Horvath et al. [50] use correlation EM analysis to recover the weights and biases in the convolutional layers of a CNN model.

Taneja et al. [132] capture side-channel leakage of frequency, power, and temperature (monitored via internal sensors) in response to the workload executed in both integrated and discrete GPUs. Their attacks, which observe instruction and data-dependent behaviour, require different privilege levels depending on vendors. For instance, Apple permits unprivileged access to both, whereas Google restricts temperature readings but allows unprivileged access to power.

5.3.2 Remote Access-Based. Zhan et al. [155] identify the DVFS feature used in GPU as the underlying cause of an MMIO side-channel vulnerability. In scenarios where a victim employs an NVIDIA or AMD GPU, an attacker can monitor the victim's activities and discern the visited webpages (i.e., a website fingerprinting attack). The key idea is capturing the fluctuating workload of a GPU during webpage rendering, which influences the MMIO signal it emits. These workload variations, coupled with the need to manage heat generation, fan noise, and minimize power consumption, lead to changes in the GPU's performance levels (e.g., P-states in NVIDIA GPUs). Consequently, when a performance level switches on or off, distinct MMIO signals corresponding to

the GPU's memory clock frequencies appear or vanish in the spectrum. Remarkably, this attack can intercept MMIO signal leakage at distances of up to 6 meters, even through a wall. Liang et al. [83] conduct a similar far-field MMIO side-channel attack to infer the number of layers and their types, the number of kernels, sizes of layers and strides in an NN model.

Takeaway points. ❶ MMIO channels are a consistent and critical leakage source, acting as a bridge between CPU and GPU for both local and remote side-channel attacks. ❷ Access to internal sensors (e.g., temperature, power) varies by vendor. On platforms like Apple, where such access is unprivileged, this creates opportunities for exploitation. ❸ Remote physical attacks are becoming increasingly feasible, with some able to succeed from several meters away, even through walls, without privileged access. This broadens the threat model to include fully remote, passive attackers.

5.3.3 Countermeasures. MMIO side channels from GPUs can be exploited when these EM signals are dependent on computations, revealing information about ongoing activities. Such side-channel signals are easily measurable when they are strong and can propagate over several meters. Two countermeasures can mitigate such vulnerabilities.

(i) *Hiding and masking* [94]: Hiding aims to reduce the **Signal to Noise Ratio (SNR)** available to attackers. Generating MMIO noise is possible, but shielding the computer to decrease the intensity of emitted MMIO signals is more effective. Hardware manufacturers could also focus on minimizing MMIO emissions in future product designs. Masking involves combining data with random mask values to ensure the observable leakage is independent of the secret. For example, creating random GPU workloads during sensitive tasks can disrupt usage patterns and reduce predictability [91, 155]. However, masking can be complicated due to the need for random masks and the added complexity of the algorithms.

(ii) *Coarsening the granularity*: Another approach is to make GPU DVFS less sensitive to workload changes. This would keep performance more stable, leading to less detailed leaked information and reducing the chance of sensitive data being inferred [155]. However, such changes might negatively affect the efficiency benefits of DVFS.

5.4 Fault Injection Attacks

5.4.1 Software-Based. Frigo et al. [37] use GPU-based timers to execute a JavaScript-based side-channel attack, enabling attackers to identify contiguous regions of physical memory. This method allows for remote triggering of Rowhammer bit flips in the identified memory areas. Consequently, an attacker can breach **Address Space Layout Randomization (ASLR)** and evade the Firefox sandbox on Android platforms featuring integrated Qualcomm Adreno GPUs. Sabbagh et al. [119] introduced the first non-invasive fault attack named “overdrive” on a GPU. By exploiting the DVFS interface, an adversary can configure out-of-specification voltage and frequency combinations on the host CPU and send malicious overdrive commands to the GPU control registers, inducing random faults during GPU kernel execution. They also perform a misclassification attack on victim CNN inference [120], improving the overdrive attack by addressing its limitations in timing precision and the lack of analysis on how fault injection affects silent data corruption.

However, unlike encryption algorithms, where hardware fault injection attacks are highly effective, ML models, especially well-trained commercial DNN models, are inherently more fault-tolerant. This is because computation errors from random hardware faults are usually absorbed by later layers and do not greatly impact accuracy. Therefore, inducing hardware faults at specific timing points, optimizing the glitch voltage-frequency pair, and determining the appropriate fault duration are critical for generating controllable and impactful faults in ML models. To this end, Sun et al. [130] focus on searching sensitive targets of a CNN model, identifying proper parameters for timing and position, refining fault injection parameters, and ultimately reducing inference

accuracy by an average of 69.1%. dos Santos et al. [28] propose a two-level fault injection framework combining the accuracy of **Register Transfer Level (RTL)** fault injection with the efficiency of software fault injection.

Overall, researchers have primarily leveraged software interfaces to inject faults, often utilizing DVFS or Rowhammer attacks. Rowhammer attacks exploit the DRAM vulnerability where repeated accesses to a memory row (i.e., hammering) can cause bit flips in adjacent rows, enabling dangerous exploits like privilege escalation from a normal user to a system administrator. The high parallelism and memory bandwidth inherent in GPUs potentially make them more vulnerable to Rowhammer attacks, as they can generate memory accesses faster than CPUs.

Takeaway points. ❶ Fault injection ‘can be triggered entirely through software by abusing out-of-spec voltage-frequency settings, exposing a lack of safeguards in GPU power management and its tight coupling with CPU controls. ❷ While DNNs are generally resilient to random faults, targeted injection at specific layers or operations can significantly reduce accuracy. Achieving this, however, requires precise control over timing, location, and fault parameters. ❸ The high memory bandwidth and parallelism of GPUs make them especially prone to Rowhammer attacks, as they can generate rapid memory access patterns that increase the risk of bit flips.

5.4.2 Countermeasures. Protections against Rowhammer exploits are most effective when implemented in the main memory or the memory controller. Frigo et al. [37] investigate preventing attackers from targeting valuable data by enforcing stricter memory reuse policies. One solution involves enhancing the physical compartmentalization suggested by CATT [10] to user-space applications. However, this approach brings tradeoffs in complexity, performance, and capacity, since dynamically allocating (isolated) pages increases complexity and can have performance implications.

Regarding DVFS-based attacks, it is feasible to make it more difficult for adversaries to control and exploit the interfaces. Voltage and clock signal monitors can be used to detect unconventional changes¹. Additionally, some vendors have disabled all software-accessible interfaces for DVFS. For example, Intel defends against DVFS-based attacks on Intel SGX [103] with a BIOS setting to disable the overclocking mailbox interface and with a microcode update that includes the current state of this setting in the SGX TCB attestation. A similar strategy could also be applied to GPUs.

6 Discussions on Future Directions in GPU TEE

In Section 4, we provided a taxonomy of existing GPU TEE designs, highlighting their architectural differences, security assumptions, and threat models. Building on this foundation, Sections 5 and the supplemental material (due to the limited space of this article) reviewed and compared a broad range of attacks targeting standalone GPUs and established CPU TEEs such as Intel SGX, Arm TrustZone, and AMD SEV. In this section, we extrapolate from these findings to identify potential attack vectors that may arise in the context of CPU TEEs, mapping them to the specific components and trust boundaries involved in these systems. We note that as GPU TEEs are not widely commercially available, it is not yet possible to practically evaluate these attack vectors, but we believe that according vulnerabilities might be discovered in the future.

6.1 Potential Attacks Against x86-Based GPU TEE Prototype

As summarized in Section 4, the state-of-the-art designs targeting x86 architecture each have unique characteristics aimed at reducing the TCB size and minimizing hardware modifications. The

¹cf. e.g., Intel’s approach: <https://www.intel.com/content/www/us/en/newsroom/news/the-story-behind-new-intel-security-feature.html>

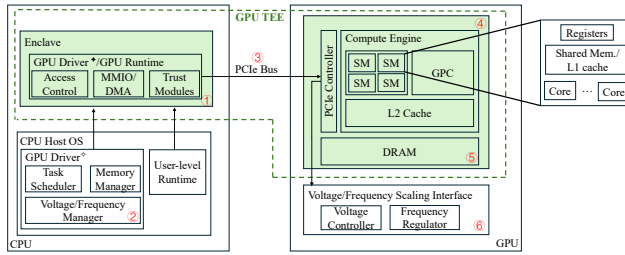


Fig. 6. Potential attacks against x86-based GPU TEE prototype.

primary goal of these designs is to protect GPU resources from a compromised OS, including the GPU driver. We outline the key design ideas and provide a general overview in Figure 6. Specifically, in this threat model, the adversary controls the entire software stack, encompassing OS, hypervisor, and device drivers. The adversary also aims to compromise the hardware and software I/O data path between a user application and the GPU. The functionalities of GPU drivers are often partitioned into secure and non-secure parts instead of porting the entire driver into the TEE enclave, which would increase the TCB size. Given that the adversary can compromise the OS, the GPU driver located in the OS is also within this scope. The device memory of the GPU is trusted and the adversary cannot observe or corrupt the data stored in it. Moreover, we recognize that the power and frequency management modules residing in both the CPU and GPU sides are outside the scope of protection. Next, based on this security model, we will analyze potential attacks that could pose threats and leak information when such designs are deployed in real-life applications.

6.1.1 Existing Attacks against Intel SGX. Given that the essential GPU resource management functions within the GPU driver reside within the TEE enclave, the majority of attacks outlined in the supplemental material could potentially target this enclave to glean secrets (cf. attack surface ① in Figure 6). It is worth noting that while some research may assert that most side-channel attacks against SGX are out of scope, it is critical to provide the corresponding protection when considering the deployment of GPU TEE designs in real-world cloud platforms (where even some forms of physical access by malicious insiders might be relevant). These potential leakages can be categorized as follows: (i) Observing memory access patterns allows attackers to deduce the control flow of programs within the SGX enclave, including decision-making processes. Armed with this insight, attackers could monitor GPU commands and data transfers, thereby inferring the tasks executed on the GPU. For instance, it might become feasible to infer the quantity (and size) of detected objects in each frame of a CNN-based video analysis task [115]. (ii) Monitoring page fault patterns enables attackers to extract cryptographic keys from implementations of certain cryptographic open-source software libraries (e.g., OpenSSL and Libcrypt) [128]. With this knowledge, attackers could subsequently access all sensitive information transmitted to the GPU. (iii) Exploiting timing channels allows attackers to infer sensitive information about kernels running on the GPU. For instance, Telekine [52] demonstrates how attackers can accurately classify images from ImageNet by solely observing the timing of GPU kernel execution, without needing access to the images themselves.

6.1.2 DVFS-Based Attacks. To enhance energy efficiency, many current CPU processors enable the DVFS extension, which adjusts the frequency and voltage of processor cores based on real-time computational loads. This capability now extends to accelerators like NVIDIA GPUs. As demonstrated by Lightning [130], NVIDIA GPU working voltages can be altered through software instructions executed on the CPU via NVIDIA drivers, potentially enabling attackers to induce transient voltage glitches for fault injection attacks. Furthermore, current NVIDIA GPUs offer

unrestricted control for **Phase Locked Loop (PLL)** and clock divider, allowing easy overclocking by adjusting the frequency offset using NVIDIA, thus potentially facilitating frequency-based fault injection attacks. Consequently, for a x86-based GPU TEE prototype, we identify two attack surfaces (cf. ② and ⑥) regarding DVFS-based attacks in Figure 6.

On the CPU side. In many existing GPU TEEs prototypes [57, 58, 75, 140, 154, 162], the part of the GPU driver responsible for DVFS parameter control lies outside their TCB. Consequently, an attacker can flexibly adjust GPU voltage and frequency from the CPU side to induce faults on the GPU side. Additionally, it warrants investigation whether previous attacks against SGX could gain access to the relevant memory region to alter DVFS parameters, even if the GPU driver managing these parameters is protected by running in an SGX enclave.

On the GPU side. Due to the absence of protection on GPU hardware, attackers with physical access to GPU hardware could directly manipulate GPU voltage, injecting faults into running GPU kernels. For instance, “VoltPillager” [18] can inject messages on the **Serial Voltage Identification (SVID)** bus between the CPU and the voltage regulator on the motherboard, enabling precise control over CPU core voltage. Ultimately, this jeopardizes the confidentiality and integrity of Intel SGX enclaves. Exploring whether attackers can inject malicious SVID packets (or similar) into the respective bus on the GPU card presents an intriguing avenue for further investigation.

6.1.3 Physical Attacks on PCIe Bus and GPU Memory. In many existing GPU TEEs prototypes lacking vendor hardware support (cf. attack surfaces ③ and ⑤ in Figure 6), GPUs lack a trusted memory region, leaving data in GPU memory exposed in plaintext. Consequently, direct physical access to GPU memory can compromise sensitive data. Further, cold boot attacks [47] could be relevant, if an adversary can ensure (e.g., by cooling) that TEE-protected memory contents are retained and can be readout after a reset of the GPU.

Additionally, as PCIe interconnects are exposed, injecting malicious PCIe packets via specialized hardware is feasible. Simply securing the routing path to the GPU is not sufficient to protect GPU control via MMIO or DMA [58]. While most designs employ authenticated encryption to ensure data integrity during transfer, this alone is insufficient to protect data confidentiality and integrity. For instance, certain designs lack a hardware root of trust necessary for secure authentication and establishment of trusted communication channels, involving secure storage for cryptographic keys and secure execution environments for cryptographic operations. Furthermore, ensuring secure boot and firmware integrity is crucial to guarantee the correctness of the secure channel establishment process.

6.1.4 Architectural/Logical Attacks on GPU. Previous attacks on memory vulnerabilities (see Section 5.1) are also possible, as current GPU TEE designs often overlook memory clearing between different GPU contexts (which is also relevant across resets in the case of cold boot attacks mentioned above). Therefore, a secure memory management strategy, including memory initialization and clearance, is critical to the integrity and security of the GPU TEE workflow.

6.2 Potential Attacks against NVIDIA Hopper H100-Based GPU TEE

In addition to the state-of-the-art x86-based GPU TEE prototypes, NVIDIA’s Hopper H100 Tensor Core GPU, based on the Hopper architecture, recently introduces advanced features for confidential computing (only available to select users at present). According to NVIDIA’s whitepaper [109], the H100 relies on specific CPU TEEs, such as Intel TDX, AMD SEV-SNP, or Arm CCA, to enable confidential computing. Here, we provide a general system model of an NVIDIA Hopper H100-based GPU TEE based on the whitepaper in Figure 7. Specifically, *on the CPU side*, this GPU TEE operates within the standard threat model of VM-based TEEs [121, 122, 158]. The untrusted hypervisor loads

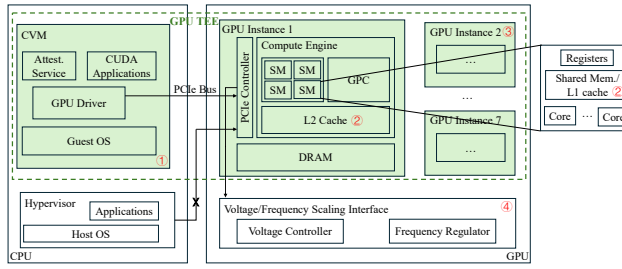


Fig. 7. Potential attacks against NVIDIA Hopper H100-based GPU TEE.

the VM image into memory and manages the initial configurations. Remote attestation measures the VM's initial memory state before initiating the boot process. The software executing within the VM (including the guest OS, user applications, and trusted modules for TEEs) is part of the TCB. The specifications require the hypervisor to set up certain initial states (e.g., the number of vCPUs, supported hardware features, and memory size). The hardware verifies these configurations and only enters the VM if the setup is correct. Additionally, the hardware zeroes out certain values (e.g., specific general-purpose registers) before exiting the VM and manages context saving and restoring during VM context switches. *On the GPU side*, the H100 GPU is expected to feature a memory encryption engine (most likely an **Memory Encryption Engine (MEE)** [22] similar to SGX) and other capabilities to ensure the confidentiality and integrity of data and code, even defending against basic physical attacks targeting the PCIe bus and GPU memory. However, the whitepaper does not provide detailed technical information about these features.

6.2.1 Existing Attacks against VM-Based TEEs. Perhaps because VM-based TEEs have only been proposed in recent years, there is currently limited research on attacks against them. However, recent developments have shown that software-based fault injection attacks can be highly effective against AMD SEV-SNP. These attacks would allow an adversary to compromise the GPU driver within the VM, enabling precise control over sensitive tasks running on the GPU (cf. attack surface ① in Figure 7). For example, the “WeSee” attack [121] targets AMD SEV-SNP used in NVIDIA H100-based GPU TEEs, allowing an attacker to perform arbitrary memory writes by faking a #VC for MMIO reads. This capability would enable the attacker to modify commands and data sent to the GPU. More critically, attacks like CacheWarp [158], which remain unmitigated in the latest SEV-SNP implementation, do not depend on the specifics of the guest VM. Therefore, it is crucial to equip NVIDIA H100-based GPU TEEs with protection against these attacks to preserve the confidentiality and integrity of GPU data. It is also important to explore potential attacks against other VM-based TEEs, such as Intel TDX, as these are integral to NVIDIA H100-based GPU TEEs.

6.2.2 DVFS-Based Attacks. Similar to GPU TEE prototypes, we identify two attack surfaces (cf. ① and ④) regarding DVFS-based attacks in Figure 7.

On the CPU side. Recall that existing attacks targeting VM-based TEEs (e.g., AMD SEV-SNP) allow attackers to escalate privileges to root and write any value of their choice to any location in the victim VM. Thus, attackers with these abilities might be able to manipulate the GPU's voltage and frequency using software instructions performed on the CPU via the NVIDIA driver.

On the GPU side. The H100 whitepaper [111] lacks detail about voltage and frequency management. Assuming an attacker has physical access, they could potentially execute attacks like “VoltPillager” [18] to inject malicious SVID packets or equivalent into the relevant GPU card's bus, thereby altering the GPU voltage.

In summary, an attacker with the described capabilities could modify the voltage and frequency from both the CPU and GPU sides. However, it remains uncertain whether and how these changes affect tasks running on the NVIDIA H100 GPU. Specifically, it is unclear if the induced faults significantly impact tasks, such as degrading ML model accuracy or fully extracting private encryption keys. At the time of writing this survey, the complete confidential workflow of the NVIDIA H100-based GPU TEE is unavailable, as the corresponding NVIDIA libraries are still under development.

6.2.3 Microarchitectural Side-Channel and Covert-Channel Attacks on GPU. The H100 whitepaper [111] emphasizes the enhanced security features of the Hopper architecture for confidential computing, particularly in collaborative multi-party computing scenarios, namely *single tenant with multiple GPUs* and *multiple tenants with a single GPU*. However, vulnerabilities discussed in Section 5.2 may potentially lead to security breaches on the NVIDIA H100 GPU.

Single tenant with multiple GPUs. In the scenario of single tenant with multiple GPUs, a single VM is concurrently assigned multiple physically independent GPUs, each connected via NVLink. Recall that Dutta et al. [31] demonstrate that an attacker on one GPU could induce contention on the L2 cache of another GPU via NVLink, enabling a covert channel and Prime+Probe attacks across GPUs. However, such attacks may face challenges in the NVIDIA H100 GPU model due to two key factors (cf. attack surface ② in Figure 7). Firstly, the attacker's process needs access to a remote GPU's memory through NVLink, which may be restricted in the threat model of the NVIDIA H100 GPU TEE. Thus, the challenge lies in how to enable the spy process to run within one of the assigned GPUs and have access to another GPU's memory. Secondly, the H100 GPU features a 50 MB L2 cache, 1.25× larger than the A100 GPU. Nevertheless, a larger L2 cache with more sets and ways can make Prime+Probe attacks more difficult, because the attacker needs to fill and monitor a larger number of cache lines.

Multiple tenants with a single GPU. In the case of multiple tenants with a single GPU (cf. attack surface ③ in Figure 7), facilitated by the MIG feature, an NVIDIA H100 GPU can be partitioned into multiple GPU instances, each isolated in terms of memory system paths, including the on-chip crossbar ports, L2 cache banks, memory controllers, and DRAM address busses. However, as discussed in Section 5.2, Zhang et al. [159] demonstrate a design flaw of the MIG feature where the last-level TLB is not securely partitioned by all GPU instances. Consequently, this flaw enables covert-channel attacks between different GPU instances. Given that this mode supports multiple tenants, it is pertinent to investigate whether each GPU instance created with MIG truly has separate and isolated paths through other microarchitectural components such as L1 cache, warp scheduler, and command processor.

6.3 Potential Attacks against Arm-Based GPU TEE Prototype

Due to various architectural disparities in TEEs for Arm GPUs, several studies have explored Arm-based GPU TEEs. In Figure 8, we base it on the design proposed by StrongBox [26]. StrongBox serves as a suitable foundation as it was recently introduced and does not rely on specific Arm endpoint features or necessitate hardware modifications to GPU or CPU chips. Notably, while secure GPU computing on Arm or Intel platforms may not align with Arm's CCA realm-style architecture [141], we omit insights regarding this, as it closely resembles that of NVIDIA Hopper H100-based GPU TEE. In Figure 8, the attacker gains control over the kernel and the entire GPU software stack, including the GPU driver and runtime. This control enables them to manipulate sensitive data and code within GPU applications. They can directly access unified memory used for GPU tasks or manipulate peripherals to evade detection. Furthermore, by compromising the GPU driver, the attacker can undermine the memory management of GPU applications, potentially mapping sensitive data to unprotected regions. Similar to existing x86-based GPU TEEs, trust is

elements, including both CPU and GPU. Thus, as depicted in Figure 8 (cf. attack surface ⑤), attackers with privileges can potentially induce hardware faults by manipulating processor voltage and frequency settings through the PMIC. Additionally, assuming an attacker gains physical access to the voltage regulator or clock control module within the **System on Chip (SoC)**, this prompts further investigation into whether hardware-based attacks targeting these components [18] are also applicable to Arm GPUs.

6.4 Comparison of Attacks on Different GPU TEEs

x86-based GPU TEE prototypes, NVIDIA Hopper H100, and Arm-based GPU TEEs adopt distinct trust models, hardware assumptions, and protection strategies. x86-based designs (e.g., Graviton, HIX, HETEE) typically build on Intel SGX, placing only essential GPU driver components inside the enclave to minimize the TCB. These prototypes often lack GPU-side memory encryption and leave out DVFS and power management from protection, making them susceptible to software-based fault injection and physical attacks (e.g., cold boot, SVID manipulation). In contrast, NVIDIA Hopper H100 integrates vendor-supported confidential computing features, including memory encryption and compatibility with VM-based TEEs. While this offers system-wide isolation, it inherits known vulnerabilities of VM-based TEEs, such as software-based fault injection (e.g., WeSee) and cache manipulation (e.g., CacheWarp). Furthermore, the limited public documentation presents challenges for third-party validation and systematic defense design. Arm-based GPU TEE designs, such as StrongBox, use TrustZone to provide software-based isolation without requiring hardware modifications. This approach improves portability and ease of deployment but sacrifices granularity. These systems typically lack hardware-enforced memory encryption and rely on unified memory and shared caches, which can expose sensitive data to physical and side-channel attacks. Operating in resource-constrained environments (e.g., edge devices), Arm-based TEEs tend to have a smaller TCB but may not adequately defend against DVFS abuse or microarchitectural leakage.

In summary, existing x86-based prototypes prioritize minimal hardware changes but depend on SGX's integrity; NVIDIA H100 adopts holistic, hardware-enhanced isolation but inherits the complexity and weaknesses of VM-based TEEs; and Arm-based TEEs favor lightweight integration at the cost of fine-grained protection. From an attack perspective, x86 prototypes are vulnerable at the interface between CPU TEEs and unprotected GPUs, NVIDIA's model is exposed through VM-level fault channels and microarchitectural sharing, and Arm designs are weakest at the hardware and cache/memory system level. These architectural differences reflect distinct tradeoffs and highlight the need for architecture-aware defenses tailored to each GPU TEE design.

7 Conclusion

With the proliferation of heterogeneous systems integrating GPUs, GPU TEEs are emerging as effective solutions for ensuring confidentiality and integrity. In this survey, we first examined existing GPU TEEs prototypes and NVIDIA's first commercial GPU TEE, providing key insights into their design implications and characteristics. We reviewed recent attacks extended across various types of GPUs, including integrated GPUs, single GPUs, and multiple GPUs, along with the efforts to mitigate these threats. Additionally, we summarized popular attacks against common CPU TEEs such as Intel SGX, Arm TZ, and AMD SEV. Based on these insights, we explored potential attacks at both physical and software levels on x86-based and Arm-based GPU TEEs prototypes, as well as the NVIDIA H100 GPU, highlighting the need for careful analysis of the already-complex attack surface. With innovations in GPU architecture and isolation mechanisms in GPU TEEs, this attack surface will further expand, necessitating the evolution of secure designs and defence strategies as these systems are widely deployed across computing domains.

References

- [1] Jaeguk Ahn, Cheolgyu Jin, Jiho Kim, Minsoo Rhu, Yunsu Fei, David Kaeli, and John Kim. 2021. Trident: A hybrid correlation-collision GPU cache timing attack for AES key recovery. In *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 332–344.
- [2] Jaeguk Ahn, Jiho Kim, Hans Kasan, Leila Delshadtehrani, Wonjun Song, Ajay Joshi, and John Kim. 2021. Network-on-chip microarchitecture-based covert channel in gpus. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*. 565–577.
- [3] Fritz Alder, Lesly-Ann Daniel, David Oswald, Frank Piessens, and Jo Van Bulck. 2024. Pandora: Principled symbolic validation of Intel SGX enclave runtimes. In *45th IEEE Symposium on Security and Privacy (S&P)*.
- [4] Fritz Alder, Jo Van Bulck, David Oswald, and Frank Piessens. 2020. Faulty point unit: ABI poisoning attacks on Intel SGX. In *Proceedings of the 36th Annual Computer Security Applications Conference*. 415–427.
- [5] AMD. 2023. *AMD Secure Encrypted Virtualization (SEV)*. Retrieved from <https://www.amd.com/en/developer/sev.html>
- [6] Arm. 2004. *ARM Security Technology: Building a Secure System using TrustZone Technology*. Retrieved from <https://www.arm.com/products/security-on-arm/trustzone>
- [7] Arm. 2023. *Arm Confidential Compute Architecture*. Retrieved from <https://www.arm.com/architecture/security-features/arm-confidential-compute-architecture>
- [8] Arm. 2023. *Arm Realm Management Extension (RME) System Architecture*. Retrieved from <https://developer.arm.com/documentation/den0129/latest/>
- [9] Gal Beniamini. 2016. TrustZone Kernel Privilege Escalation (CVE-2016-2431). Retrieved from <https://bits-please.blogspot.com/2016/06/trustzone-kernel-privilege-escalation.html>
- [10] Ferdinand Brasser, Lucas Davi, David Gens, Christopher Liebchen, and Ahmad-Reza Sadeghi. 2017. Can't touch this: Software-only mitigation against rowhammer attacks targeting kernel memory. In *26th USENIX Security Symposium (USENIX Security 17)*. 117–130.
- [11] Ferdinand Brasser, Urs Müller, Alexandra Dmitrienko, Kari Kostiainen, Srdjan Capkun, and Ahmad-Reza Sadeghi. 2017. Software grand exposure: SGX cache attacks are practical. In *11th USENIX Workshop on Offensive Technologies (WOOT 17)*.
- [12] Robert Buhren, Shay Gueron, Jan Nordholz, Jean-Pierre Seifert, and Julian Vetter. 2017. Fault attacks on encrypted general purpose compute platforms. In *Proceedings of the Seventh ACM on Conference on Data and Application Security and Privacy*. 197–204.
- [13] Robert Buhren, Hans-Niklas Jacob, Thilo Krachenfels, and Jean-Pierre Seifert. 2021. One glitch to rule them all: Fault injection attacks against amd's secure encrypted virtualization. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*. 2875–2889.
- [14] Pierre Carru. 2017. *Attack TrustZone using Rowhammer*. Retrieved from https://grehack.fr/data/2017/slides/GreHack17_Attack_TrustZone_with_Rowhammer.pdf
- [15] David Cerdeira, Nuno Santos, Pedro Fonseca, and Sandro Pinto. 2020. Sok: Understanding the prevailing security vulnerabilities in trustzone-assisted tee systems. In *2020 IEEE Symposium on Security and Privacy (SP)*. IEEE, 1416–1432.
- [16] Guoxing Chen, Sanchuan Chen, Yuan Xiao, Yinqian Zhang, Zhiqiang Lin, and Ten H Lai. 2019. Sgxpectre: Stealing intel secrets from sgx enclaves via speculative execution. In *2019 IEEE European Symposium on Security and Privacy (EuroS&P)*. IEEE, 142–157.
- [17] Zitai Chen and David Oswald. 2023. PMFault: Faulting and bricking server CPUs through management interfaces: Or: A modern example of halt and catch fire. *IACR Transactions on Cryptographic Hardware and Embedded Systems* 2023, 2 (Mar. 2023), 1–23.
- [18] Zitai Chen, Georgios Vasilakis, Kit Murdock, Edward Dean, David Oswald, and Flavio D Garcia. 2021. VoltPillager: Hardware-based fault injection attacks against Intel SGX enclaves using the SVID voltage scaling interface. In *30th USENIX Security Symposium (USENIX Security 21)*. 699–716.
- [19] Łukasz Chmielewski and Léo Weissbart. 2021. On reverse engineering neural network implementation on GPU. In *Applied Cryptography and Network Security Workshops: ACNS 2021 Satellite Workshops, AIBlock, AIHWS, AIoTS, CIMSS, Cloud S&P, SCI, SecMT, and SiMLA, Kamakura, Japan, June 21–24, 2021, Proceedings*. Springer, 96–113.
- [20] Haehyun Cho, Penghui Zhang, Donguk Kim, Jinbum Park, Choong-Hoon Lee, Ziming Zhao, Adam Doupe, and Gail-Joon Ahn. 2018. Prime+ count: Novel cross-world covert channels on arm trustzone. In *Proceedings of the 34th Annual Computer Security Applications Conference*. 441–452.
- [21] Tobias Cloosters, Michael Rodler, and Lucas Davi. 2020. TeeRex: Discovery and exploitation of memory corruption vulnerabilities in SGX enclaves. In *29th USENIX Security Symposium (USENIX Security 20)*. 841–858.
- [22] Intel Corporation. 2015. *Intel R © Software Guard Extensions*. Retrieved from https://community.intel.com/legacyfs/online/drupal_files/332680-002.pdf
- [23] Victor Costan and Srinivas Devadas. 2016. Intel SGX explained. *Cryptology ePrint Archive* (2016).

- [24] Patrick Cronin, Xing Gao, Haining Wang, and Chase Cotton. 2021. An exploration of ARM system-level cache and GPU side channels. In *Proceedings of the 37th Annual Computer Security Applications Conference*. 784–795.
- [25] Fergus Dall, Gabrielle De Micheli, Thomas Eisenbarth, Daniel Genkin, Nadia Heninger, Ahmad Moghimi, and Yuval Yarom. 2018. Cachequote: Efficiently recovering long-term secrets of SGX EPID via cache attacks. *IACR Transactions on Cryptographic Hardware and Embedded Systems* 2018, 2 (2018), 171–191. DOI: <https://doi.org/10.13154/tches.v2018.i2.171-191>
- [26] Yunjie Deng, Chenxu Wang, Shunchang Yu, Shiqing Liu, Zhenyu Ning, Kevin Leach, Jin Li, Shoumeng Yan, Zhengyu He, Jiannong Cao, et al. 2022. Strongbox: A gpu tee on arm endpoints. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*. 769–783.
- [27] Bang Di, Daokun Hu, Zhen Xie, Jianhua Sun, Hao Chen, Jinkui Ren, and Dong Li. 2021. TLB-pilot: Mitigating TLB contention attack on GPUs with microarchitecture-aware scheduling. *ACM Transactions on Architecture and Code Optimization (TACO)* 19, 1 (2021), 1–23.
- [28] Fernando F dos Santos, Josie E Rodriguez Condia, Luigi Carro, Matteo Sonza Reorda, and Paolo Rech. 2021. Revealing gpus vulnerabilities by combining register-transfer and software-level fault injection. In *2021 51st Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. IEEE, 292–304.
- [29] Zhao-Hui Du, Zhiwei Ying, Zhenke Ma, Yufei Mai, Phoebe Wang, Jesse Liu, and Jesse Fang. 2017. Secure encrypted virtualization is insecure. arXiv:1712.05090. Retrieved from <https://arxiv.org/abs/1712.05090>
- [30] Sankha Baran Dutta, Hoda Naghibijouybari, Nael Abu-Ghazaleh, Andres Marquez, and Kevin Barker. 2021. Leaky buddies: Cross-component covert channels on integrated cpu-gpu systems. In *ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 972–984.
- [31] Sankha Baran Dutta, Hoda Naghibijouybari, Arjun Gupta, Nael Abu-Ghazaleh, Andres Marquez, and Kevin Barker. 2023. Spy in the GPU-box: Covert and side channel attacks on multi-GPU systems. In *Proceedings of the 50th Annual International Symposium on Computer Architecture*. 1–13.
- [32] Dmitry Evtvushkin, Dmitry Ponomarev, and Nael Abu-Ghazaleh. 2016. Jump over ASLR: Attacking branch predictors to bypass ASLR. In *2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 1–13.
- [33] Dmitry Evtvushkin, Dmitry Ponomarev, and Nael Abu-Ghazaleh. 2016. Understanding and mitigating covert channels through branch predictors. *ACM Transactions on Architecture and Code Optimization (TACO)* 13, 1 (2016), 1–23.
- [34] Dmitry Evtvushkin, Ryan Riley, Nael CSE Abu-Ghazaleh, ECE, and Dmitry Ponomarev. 2018. Branchscope: A new side-channel attack on directional branch predictor. *ACM SIGPLAN Notices* 53, 2 (2018), 693–707.
- [35] Shengyu Fan, Zhiwei Wang, Weizhi Xu, Rui Hou, Dan Meng, and Mingzhe Zhang. 2023. Tensorfhe: Achieving practical computation on encrypted data using gpgpu. In *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 922–934.
- [36] Shufan Fei, Zheng Yan, Wenxiu Ding, and Haomeng Xie. 2021. Security vulnerabilities of SGX and countermeasures: A survey. *ACM Computing Surveys (CSUR)* 54, 6 (2021), 1–36.
- [37] Pietro Frigo, Cristiano Giuffrida, Herbert Bos, and Kaveh Razavi. 2018. Grand pwning unit: Accelerating microarchitecture attacks with the GPU. In *2018 IEEE Symposium on Security and Privacy (sp)*. IEEE, 195–210.
- [38] Yiwen Gao, Hailong Zhang, Wei Cheng, Yongbin Zhou, and Yuchen Cao. 2018. Electro-magnetic analysis of GPU-based AES implementation. In *Proceedings of the 55th Annual Design Automation Conference*. 1–6.
- [39] Tara Ghasempouri, Jaan Raik, Cezar Reinbrecht, Said Hamdioui, and Mottaqiallah Taouil. 2023. Survey on architectural attacks: A unified classification and attack model. *Comput. Surveys* 56, 2 (2023), 1–32.
- [40] Lukas Giner, Roland Czerny, Christoph Gruber, Fabian Rauscher, Andreas Kogler, Daniel De Almeida Braga, and Daniel Gruss. 2024. Generic and automated drive-by GPU cache attacks from the browser. In *Proceedings of the 19th ACM Asia Conference on Computer and Communications Security (AsiaCCS'24)*. Association for Computing Machinery, 128–140. DOI: <https://doi.org/10.1145/3634737.3656283>
- [41] Johannes Götzfried, Moritz Eckert, Sebastian Schinzel, and Tilo Müller. 2017. Cache attacks on intel SGX. In *Proceedings of the 10th European Workshop on Systems Security*. 1–6.
- [42] Daniel Gruss, Clémentine Maurice, Klaus Wagner, and Stefan Mangard. 2016. Flush+ flush: A fast and stealthy cache attack. In *Detection of Intrusions and Malware, and Vulnerability Assessment: 13th International Conference, DIMVA 2016, Spain, July 7-8, 2016*. Springer, 279–299.
- [43] Daniel Gruss, Raphael Spreitzer, and Stefan Mangard. 2015. Cache template attacks: Automating attacks on inclusive Last-Level caches. In *24th USENIX Security Symposium (USENIX Security 15)*. 897–912.
- [44] Roberto Guanciale, Hamed Nemat, Christoph Baumann, and Mads Dam. 2016. Cache storage channels: Alias-driven attacks and verified countermeasures. In *2016 IEEE Symposium on Security and Privacy (SP)*. IEEE, 38–55.
- [45] Roberto Guanciale, Nicolae Paladi, and Arash Vahidi. 2022. SoK: Confidential quartet-comparison of platforms for virtualization-based confidential computing. In *2022 IEEE International Symposium on Secure and Private Execution Environment Design (SEED)*. IEEE, 109–120.

- [46] Jago Gyselinck, Jo Van Bulck, Frank Piessens, and Raoul Strackx. 2018. Off-limits: Abusing legacy x86 memory segmentation to spy on enclaved execution. In *Engineering Secure Software and Systems: 10th International Symposium, ESSoS 2018, France, June 26-27, 2018*. Springer, 44–60.
- [47] J. Alex Halderman, Seth D. Schoen, Nadia Heninger, William Clarkson, William Paul, Joseph A. Calandrino, Ariel J. Feldman, Jacob Appelbaum, and Edward W. Felten. [n. d.]. Lest we remember: Cold boot attacks on encryption keys. In *17th USENIX Security Symposium (USENIX Security 08)*.
- [48] Wenjian HE, Wei Zhang, Sharad Sinha, and Sanjeev Das. 2020. Igpu leak: An information leakage vulnerability on intel integrated gpu. In *2020 25th Asia and South Pacific Design Automation Conference (ASP-DAC)*. IEEE, 56–61.
- [49] Felicitas Hetzelt and Robert Buhren. 2017. Security analysis of encrypted virtual machines. *ACM SIGPLAN Notices* 52, 7 (2017), 129–142.
- [50] Peter Horvath, Lukasz Chmielewski, Leo Weissbart, Lejla Batina, and Yuval Yarom. 2023. BarraCUDA: Bringing electromagnetic side channel into play to steal the weights of neural networks from NVIDIA GPUs. arXiv:2312.07783. Retrieved from <https://arxiv.org/abs/2312.07783>
- [51] Xing Hu, Ling Liang, Shuangchen Li, Lei Deng, Pengfei Zuo, Yu Ji, Xinfeng Xie, Yufei Ding, Chang Liu, Timothy Sherwood, et al. 2020. Deepsniffer: A dnn model extraction framework based on learning architectural hints. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*. 385–399.
- [52] Tyler Hunt, Zhipeng Jia, Vance Miller, Ariel Szekely, Yige Hu, Christopher J Rossbach, and Emmett Witchel. 2020. Telekine: Secure computing with cloud GPUs. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*. 817–833.
- [53] Tianlin Huo, Xiaoni Meng, Wenhao Wang, Chunliang Hao, Pei Zhao, Jian Zhai, and Mingshu Li. 2020. Bluthunder: A 2-level directional predictor based side-channel attack against sgx. *IACR Transactions on Cryptographic Hardware and Embedded Systems* (2020), 321–347.
- [54] Intel. 2023. *Intel Multi-Key Total Memory Encryption*. Retrieved from <https://www.intel.com/content/dam/www/central-libraries/us/en/documents/2022-10/intel-total-memory-encryption-multi-key-whitepaper.pdf>
- [55] Intel. 2023. *Intel Trust Domain Extensions (Intel TDX)*. Retrieved from <https://www.intel.com/content/www/us/en/developer/tools/trust-domain-extensions/overview.html>
- [56] Intel. 2024. *Intel Mesa 3D Graphics Library*. Retrieved from <https://docs.mesa3d.org/systems.html>
- [57] Andrei Ivanov, Benjamin Rothenberger, Arnaud Dethise, Marco Canini, Torsten Hoeffler, and Adrian Perrig. 2023. SAGE: Software-based attestation for GPU execution. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*. 485–499.
- [58] Insu Jang, Adrian Tang, Taehoon Kim, Simha Sethumadhavan, and Jaehyuk Huh. 2019. Heterogeneous isolated execution for commodity gpus. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*. 455–468.
- [59] Yeongjin Jang, Jaehyuk Lee, Sangho Lee, and Taesoo Kim. 2017. SGX-Bomb: Locking down the processor via rowhammer attack. In *Proceedings of the 2nd Workshop on System Software for Trusted Execution*. 1–6.
- [60] Neha Jawalkar, Kanav Gupta, Arkaprava Basu, Nishanth Chandran, Divya Gupta, and Rahul Sharma. 2024. Orca: FSS-based secure training and inference with GPUs. In *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE, 597–616.
- [61] Jianyu Jiang, Ji Qi, Tianxiang Shen, Xusheng Chen, Shixiong Zhao, Sen Wang, Li Chen, Gong Zhang, Xiapu Luo, and Heming Cui. 2022. CRONUS: Fault-isolated, secure and high-performance heterogeneous computing for trusted execution environment. In *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 124–143.
- [62] Zhen Hang Jiang, Yunsu Fei, and David Kaeli. 2016. A complete key recovery timing attack on a GPU. In *2016 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 394–405.
- [63] Zhen Hang Jiang, Yunsu Fei, and David Kaeli. 2017. A novel side-channel timing attack on GPUs. In *Proceedings of the on Great Lakes Symposium on VLSI 2017*. 167–172.
- [64] Gurunath Kadam, Danfeng Zhang, and Adwait Jog. 2018. Rcoal: Mitigating gpu timing attack via subwarp-based randomized coalescing techniques. In *2018 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 156–167.
- [65] Gurunath Kadam, Danfeng Zhang, and Adwait Jog. 2020. Bcoal: Bucketing-based memory coalescing for efficient and secure gpus. In *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 570–581.
- [66] Zijo Kenjar, Tommaso Frassetto, David Gens, Michael Franz, and Ahmad-Reza Sadeghi. 2020. VOLTpwn: Attacking x86 processor integrity from software. In *29th USENIX Security Symposium (USENIX Security 20)*. 1445–1461.
- [67] Mustakimur Rahman Khandaker, Yueqiang Cheng, Zhi Wang, and Tao Wei. 2020. COIN attacks: On insecurity of enclave untrusted interfaces in SGX. In *Proceedings of the 25th International Conference on Architectural Support for Programming Languages and Operating Systems*. 971–985.
- [68] Ashfaq A. Khokhar, Viktor K. Prasanna, Muhammad E. Shaaban, and C-L Wang. 1993. Heterogeneous computing: Challenges and opportunities. *Computer* 26, 6 (1993), 18–27.

- [69] Deokjin Kim, Daehee Jang, Minjoon Park, Yunjong Jeong, Jonghwan Kim, Seokjin Choi, and Brent Byunghoon Kang. 2019. SGX-LEGO: Fine-grained SGX controlled-channel attack and its countermeasure. *Computers & Security* 82 (2019), 118–139.
- [70] Paul Kocher, Jann Horn, Anders Fogh, Daniel Genkin, Daniel Gruss, Werner Haas, Mike Hamburg, Moritz Lipp, Stefan Mangard, Thomas Prescher, et al. 2020. Spectre attacks: Exploiting speculative execution. *Commun. ACM* 63, 7 (2020), 93–101.
- [71] Daniel Komaromy. 2018. *Unbox Your Phone*. Retrieved from <https://medium.com/taszsec/unbox-your-phone-part-i-331bbf44c30c>
- [72] Jingfei Kong, Onur Aciicmez, Jean-Pierre Seifert, and Huiyang Zhou. 2009. Hardware-software integrated approaches to defend against software cache-based side channel attacks. In *2009 IEEE 15th International Symposium on High Performance Computer Architecture*. IEEE, 393–404.
- [73] Esmaeil Mohammadian Koruyeh, Khaled N Khasawneh, Chengyu Song, and Nael Abu-Ghazaleh. 2018. Spectre returns! speculation attacks using the return stack buffer. In *12th USENIX Workshop on Offensive Technologies (WOOT 18)*.
- [74] Dayeol Lee, Dongha Jung, Ian T. Fang, Chia che Tsai, and Raluca Ada Popa. 2020. An off-chip attack on hardware enclaves via the memory bus. In *29th USENIX Security Symposium (USENIX Security '20)*. 487–504.
- [75] Sunho Lee, Jungwoo Kim, Seonjin Na, Jongse Park, and Jaehyuk Huh. 2022. Tnpu: Supporting trusted execution with tree-less integrity protection for neural processing unit. In *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 229–243.
- [76] Sangho Lee, Youngsok Kim, Jangwoo Kim, and Jong Kim. 2014. Stealing webpages rendered on your browser by exploiting GPU vulnerabilities. In *2014 IEEE Symposium on Security and Privacy*. IEEE, 19–33.
- [77] Sangho Lee, Ming-Wei Shih, Prasun Gera, Taesoo Kim, Hyesoon Kim, and Marcus Peinado. 2017. Inferring fine-grained control flow inside SGX enclaves with branch shadowing. In *26th USENIX Security Symposium (USENIX Security 17)*. 557–574.
- [78] Mengyuan Li. 2022. *Understanding and Exploiting Design Flaws of AMD Secure Encrypted Virtualization*. The Ohio State University.
- [79] Mengyuan Li, Yuheng Yang, Guoxing Chen, Mengjia Yan, and Yinqian Zhang. 2024. SoK: Understanding designs choices and pitfalls of trusted execution environments. In *Proceedings of the 19th ACM Asia Conference on Computer and Communications Security (AsiaCCS'24)*. Association for Computing Machinery. DOI:<https://doi.org/10.1145/3634737.3644993>
- [80] Mengyuan Li, Yinqian Zhang, and Zhiqiang Lin. 2021. Crossline: Breaking “security-by-crash” based memory isolation in amd sev. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*. 2937–2950.
- [81] Mengyuan Li, Yinqian Zhang, Huibo Wang, Kang Li, and Yueqiang Cheng. 2021. CIPHERLEAKS: Breaking constant-time cryptography on AMDSEV via the ciphertext side channel. In *30th USENIX Security Symposium (USENIX Security 21)*. 717–732.
- [82] Mengyuan Li, Yinqian Zhang, Huibo Wang, Kang Li, and Yueqiang Cheng. 2021. TLB poisoning attacks on AMD secure encrypted virtualization. In *Proceedings of the 37th Annual Computer Security Applications Conference*. 609–619.
- [83] Sisheng Liang, Zihao Zhan, Fan Yao, Long Cheng, and Zhenkai Zhang. 2022. Clairvoyance: Exploiting far-field em emanations of gpu to “see” your dnn models through obstacles at a distance. In *2022 IEEE Security and Privacy Workshops (SPW)*. IEEE, 312–322.
- [84] Zhen Lin, Utkarsh Mathur, and Huiyang Zhou. 2019. Scatter-and-gather revisited: High-performance side-channel-resistant AES on GPUs. In *Proceedings of the 12th Workshop on General Purpose Processing Using GPUs*. 2–11.
- [85] Moritz Lipp, Daniel Gruss, Raphael Spreitzer, Clémentine Maurice, and Stefan Mangard. 2016. ARMageddon: Cache attacks on mobile devices. In *25th USENIX Security Symposium (USENIX Security 16)*. 549–564.
- [86] Moritz Lipp, Michael Schwarz, Daniel Gruss, Thomas Prescher, Werner Haas, Jann Horn, Stefan Mangard, Paul Kocher, Daniel Genkin, Yuval Yarom, et al. 2020. Meltdown: Reading kernel memory from user space. *Commun. ACM* 63, 6 (2020), 46–56.
- [87] Fangfei Liu, Qian Ge, Yuval Yarom, Frank McKeen, Carlos Rozas, Gernot Heiser, and Ruby B Lee. 2016. Catalyst: Defeating last-level cache side channel attacks in cloud computing. In *2016 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 406–418.
- [88] Fangfei Liu, Yuval Yarom, Qian Ge, Gernot Heiser, and Ruby B Lee. 2015. Last-level cache side-channel attacks are practical. In *2015 IEEE Symposium on Security and Privacy*. IEEE, 605–622.
- [89] Chao Luo, Yunsu Fei, and David Kaeli. 2019. Side-channel timing attack of RSA on a GPU. *ACM Transactions on Architecture and Code Optimization (TACO)* 16, 3 (2019), 1–18.
- [90] Chao Luo, Yunsu Fei, Pei Luo, Saoni Mukherjee, and David Kaeli. 2015. Side-channel power analysis of a GPU AES implementation. In *2015 33rd IEEE International Conference on Computer Design (ICCD)*. IEEE, 281–288.

- [91] Dina G Mahmoud, Vincent Lenders, and Mirjana Stojilović. 2022. Electrical-level attacks on CPUs, FPGAs, and GPUs: Survey and implications in the heterogeneous era. *ACM Computing Surveys (CSUR)* 55, 3 (2022), 1–40.
- [92] Haohui Mai, Jiacheng Zhao, Hongren Zheng, Yiyang Zhao, Zibin Liu, Mingyu Gao, Cong Wang, Huimin Cui, Xiaobing Feng, and Christos Kozyrakis. 2023. Honeycomb: Secure and efficient GPU executions via static validation. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*. 155–172.
- [93] Henrique Teles Maia, Chang Xiao, Dingzeyu Li, Eitan Grinspun, and Changxi Zheng. 2022. Can one hear the shape of a neural network?: Snooping the GPU via magnetic side channel. In *USENIX Security Symposium*. 4383–4400.
- [94] Stefan Mangard, Elisabeth Oswald, and Thomas Popp. 2008. *Power Analysis Attacks: Revealing the Secrets of Smart Cards*. Vol. 31. Springer Science & Business Media.
- [95] Clémentine Maurice, Christoph Neumann, Olivier Heen, and Aurélien Francillon. 2014. Confidentiality issues on a GPU in a virtualized environment. In *Financial Cryptography and Data Security: 18th International Conference, FC 2014, Christ Church, Barbados, March 3-7, 2014, Revised Selected Papers 18*. Springer, 119–135.
- [96] Pratyush Mishra, Ryan Lehmkuhl, Akshayaram Srinivasan, Wenting Zheng, and Raluca Ada Popa. 2020. Delphi: A cryptographic inference system for neural networks. In *Proceedings of the 2020 Workshop on Privacy-Preserving Machine Learning in Practice*. 27–30.
- [97] Sparsh Mittal and Jeffrey S Vetter. 2015. A survey of CPU-GPU heterogeneous computing techniques. *ACM Computing Surveys (CSUR)* 47, 4 (2015), 1–35.
- [98] Ahmad Moghimi, Gorka Irazoqui, and Thomas Eisenbarth. 2017. Cachezoom: How SGX amplifies the power of cache attacks. In *Cryptographic Hardware and Embedded Systems—CHES 2017: 19th International Conference, Taipei, Taiwan, September 25-28, 2017, Proceedings*. Springer, 69–90.
- [99] Ahmad Moghimi, Jan Wichelmann, Thomas Eisenbarth, and Berk Sunar. 2019. Memjam: A false dependency attack against constant-time crypto implementations. *International Journal of Parallel Programming* 47, 4 (2019), 538–570. DOI:<https://doi.org/10.1007/s10766-018-0611-9>
- [100] Mathias Morbitzer, Manuel Huber, Julian Horsch, and Sascha Wessel. 2018. SEVered: Subverting AMD’s virtual machine encryption. In *Proceedings of the 11th European Workshop on Systems Security*. 1–6.
- [101] Mathias Morbitzer, Sergej Proskurin, Martin Radev, Marko Dorfhuber, and Erick Quintanar Salas. 2021. Severity: Code injection attacks against encrypted virtual machines. In *2021 IEEE Security and Privacy Workshops (SPW)*. IEEE, 444–455.
- [102] David N Muchene, Klevis Luli, and Craig A Shue. 2013. Reporting insider threats via covert channels. In *2013 IEEE Security and Privacy Workshops*. IEEE, 68–71.
- [103] Kit Murdock, David Oswald, Flavio D Garcia, Jo Van Bulck, Daniel Gruss, and Frank Piessens. 2020. Plundervolt: Software-based fault injection attacks against Intel SGX. In *2020 IEEE Symposium on Security and Privacy (SP)*. IEEE, 1466–1482.
- [104] Hoda Naghibijouybari, Khaled N Khasawneh, and Nael Abu-Ghazaleh. 2017. Constructing and characterizing covert channels on gpgpus. In *Proceedings of the 50th Annual IEEE/ACM International Symposium on Microarchitecture*. 354–366.
- [105] Hoda Naghibijouybari, Esmail Mohammadian Koruyeh, and Nael Abu-Ghazaleh. 2022. Microarchitectural attacks in heterogeneous systems: A survey. *Comput. Surveys* 55, 7 (2022), 1–40.
- [106] Hoda Naghibijouybari, Ajaya Neupane, Zhiyun Qian, and Nael Abu-Ghazaleh. 2018. Rendered insecure: Gpu side channel attacks are practical. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*. 2139–2153.
- [107] Ajay Nayak, Vinod Ganapathy, and Arkaprava Basu. 2021. (Mis) managed: A novel TLB-based covert channel on GPUs. In *Proceedings of the 2021 ACM Asia Conference on Computer and Communications Security*. 872–885.
- [108] Lucien KL Ng and Sherman SM Chow. 2021. GForce: GPU-friendly oblivious and rapid neural network inference. In *30th USENIX Security Symposium (USENIX Security 21)*. 2147–2164.
- [109] NVIDIA. 2022. *NVIDIA Confidential Computing*. Retrieved from <https://www.nvidia.com/en-us/data-center/solutions/confidentialcomputing/>
- [110] NVIDIA. 2022. *NVIDIA Grace Hopper Superchip Architecture*. Retrieved from <https://resources.nvidia.com/en-us/grace-hopper-superchip>. Accessed May 2025.
- [111] NVIDIA. 2022. *NVIDIA H100 Tensor Core GPU Architecture*. Retrieved from <https://resources.nvidia.com/en-us-data-center-overview-mc/en-us-data-center-overview/gtc22-whitepaper-hopper?pfplid=17841&lb-mode=preview>
- [112] Dag Arne Osvik, Adi Shamir, and Eran Tromer. 2006. Cache attacks and countermeasures: the case of AES. In *Topics in Cryptology—CT-RSA 2006: The Cryptographers’ Track at the RSA Conference 2006, San Jose, CA, USA, February 13-17, 2005. Proceedings*. Springer, 1–20.
- [113] Roberto Di Pietro, Flavio Lombardi, and Antonio Villani. 2016. CUDA leaks: A detailed hack for CUDA and a (partial) fix. *ACM Transactions on Embedded Computing Systems (TECS)* 15, 1 (2016), 1–25.

- [114] Sandro Pinto and Nuno Santos. 2019. Demystifying arm trustzone: A comprehensive survey. *ACM Computing Surveys (CSUR)* 51, 6 (2019), 1–36.
- [115] Rishabh Poddar, Ganesh Ananthanarayanan, Srinath Setty, Stavros Volos, and Raluca Ada Popa. 2020. Visor: Privacy-preserving video analytics as a cloud service. In *29th USENIX Security Symposium (USENIX Security 20)*. 1039–1056.
- [116] Pengfei Qiu, Dongsheng Wang, Yongqiang Lyu, Ruidong Tian, Chunlu Wang, and Gang Qu. 2020. Voltjockey: A new dynamic voltage scaling-based fault injection attack on intel sgx. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 40, 6 (2020), 1130–1143.
- [117] Hany Ragab, Alyssa Milburn, Kaveh Razavi, Herbert Bos, and Cristiano Giuffrida. 2021. Crosstalk: Speculative data leaks across cores are real. In *2021 IEEE Symposium on Security and Privacy (SP)*. IEEE, 1852–1867.
- [118] Keegan Ryan. 2019. Hardware-backed heist: Extracting ECDSA keys from qualcomm’s trustzone. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*. 181–194.
- [119] Majid Sabbagh, Yunsi Fei, and David Kaeli. 2020. A novel GPU overdrive fault attack. In *2020 57th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 1–6.
- [120] Majid Sabbagh, Yunsi Fei, and David Kaeli. 2021. Gpu overdrive fault attacks on neural networks. In *2021 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*. IEEE, 1–8.
- [121] Benedict Schlüter, Supraja Sridhara, Andrin Bertschi, and Shweta Shinde. 2024. WeSee: Using malicious# VC interrupts to break AMD SEV-SNP. arXiv:2404.03526. Retrieved from <https://arxiv.org/abs/2404.03526>
- [122] Benedict Schlüter, Supraja Sridhara, Mark Kuhne, Andrin Bertschi, and Shweta Shinde. 2024. Heckler: Breaking confidential vms with malicious interrupts. In *USENIX Security*.
- [123] Michael Schwarz, Samuel Weiser, Daniel Gruss, Clémentine Maurice, and Stefan Mangard. 2017. Malware guard extension: Using SGX to conceal cache attacks. In *Detection of Intrusions and Malware, and Vulnerability Assessment: 14th International Conference, DIMVA 2017, Bonn, Germany, July 6-7, 2017, Proceedings 14*. Springer, 3–24.
- [124] AMD SEV-SNP. 2020. *Strengthening VM Isolation with Integrity Protection and More*. Advanced micro devices. Retrieved from <https://docs.amd.com/v/u/en-US/SEV-SNP-strengthening-vm-isolation-with-integrity-protection-and-more>
- [125] Ali Shafiee, Akhila Gundu, Manjunath Shevgoor, Rajeev Balasubramonian, and Mohit Tiwari. 2015. Avoiding information leakage in the memory controller with fixed service policies. In *Proceedings of the 48th International Symposium on Microarchitecture*. 89–101.
- [126] Di Shen. 2015. Exploiting trustzone on android. In *Black Hat USA 2015*. Retrieved from <https://www.blackhat.com/docs/us-15/materials/us-15-Shen-Attacking-Your-Trusted-Core-Exploiting-Trustzone-On-Android.pdf>
- [127] Shaohuai Shi, Qiang Wang, Pengfei Xu, and Xiaowen Chu. 2016. Benchmarking state-of-the-art deep learning software tools. In *2016 7th International Conference on Cloud Computing and Big Data (CCBD)*. IEEE, 99–104.
- [128] Shweta Shinde, Zheng Leong Chua, Viswesh Narayanan, and Prateek Saxena. 2016. Preventing page faults from telling your secrets. In *Proceedings of the 11th ACM on Asia Conference on Computer and Communications Security*. 317–328.
- [129] Amit Mazumder Shuvo, Tao Zhang, Farimah Farahmandi, and Mark Tehranipoor. 2023. A comprehensive survey on non-invasive fault injection attacks. Retrieved from <https://eprint.iacr.org/2023/1850>
- [130] Rihui Sun, Pengfei Qiu, Yongqiang Lyu, Jian Dong, Haixia Wang, Dongsheng Wang, and Gang Qu. 2023. Lightning: Leveraging DVFS-induced transient fault injection to attack deep learning accelerator of GPUs. *ACM Transactions on Design Automation of Electronic Systems* 29, 1 (2023).
- [131] Sijun Tan, Brian Knott, Yuan Tian, and David J Wu. 2021. CryptGPU: Fast privacy-preserving machine learning on the GPU. In *2021 IEEE Symposium on Security and Privacy (SP)*. IEEE, 1021–1038.
- [132] Hritvik Taneja, Jason Kim, Jie Jeff Xu, Stephan Van Schaik, Daniel Genkin, and Yuval Yarom. 2023. Hot pixels: Frequency, power, and temperature attacks on gpus and arm SoCs. In *32nd USENIX Security Symposium (USENIX Security 23)*. 6275–6292.
- [133] Adrian Tang, Simha Sethumadhavan, and Salvatore Stolfo. 2017. CLKSCREW: Exposing the perils of security-oblivious energy management. In *26th USENIX Security Symposium (USENIX Security 17)*. 1057–1074.
- [134] Jo Van Bulck, Marina Minkin, Ofir Weisse, Daniel Genkin, Baris Kasikci, Frank Piessens, Mark Silberstein, Thomas F Wenisch, Yuval Yarom, and Raoul Strackx. 2018. Foreshadow: Extracting the keys to the intel SGX kingdom with transient out-of-order execution. In *27th USENIX Security Symposium (USENIX Security 18)*. 991–1008.
- [135] Jo Van Bulck, David Oswald, Eduard Marin, Abdulla Aldoseri, Flavio D Garcia, and Frank Piessens. 2019. A tale of two worlds: Assessing the vulnerability of enclave shielding runtimes. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*. 1741–1758.
- [136] Jo Van Bulck, Frank Piessens, and Raoul Strackx. 2017. SGX-Step: A practical attack framework for precise enclave execution control. In *Proceedings of the 2nd Workshop on System Software for Trusted Execution*. 1–6.
- [137] Jo Van Bulck, Nico Weichbrodt, Rüdiger Kapitza, Frank Piessens, and Raoul Strackx. 2017. Telling your secrets without page faults: Stealthy page table-based attacks on enclaved execution. In *26th USENIX Security Symposium (USENIX Security 17)*. 1041–1056.

- [138] Stephan Van Schaik, Andrew Kwong, Daniel Genkin, and Yuval Yarom. 2020. SGAXe: How SGX fails in practice. Retrieved from <https://sgaxe.com/files/SGAxe.pdf>
- [139] Stephan Van Schaik, Alyssa Milburn, Sebastian Österlund, Pietro Frigo, Giorgi Maisuradze, Kaveh Razavi, Herbert Bos, and Cristiano Giuffrida. 2019. RIDL: Rogue in-flight data load. In *2019 IEEE Symposium on Security and Privacy (SP)*. IEEE, 88–105.
- [140] Stavros Volos, Kapil Vaswani, and Rodrigo Bruno. 2018. Graviton: Trusted execution environments on GPUs. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. 681–696.
- [141] Chenxu Wang, Fengwei Zhang, Yunjie Deng, Kevin Leach, Jiannong Cao, Zhenyu Ning, Shoumeng Yan, and Zhengyu He. 2024. CAGE: Complementing arm CCA with GPU extensions. In *Proceedings of the 31st Annual Network and Distributed System Security Symposium*.
- [142] Huibo Wang, Pei Wang, Yu Ding, Mingshen Sun, Yiming Jing, Ran Duan, Long Li, Yulong Zhang, Tao Wei, and Zhiqiang Lin. 2019. Towards memory safe enclave programming with rust-sgx. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*. 2333–2350.
- [143] Wenhao Wang, Guoxing Chen, Xiaorui Pan, Yinqian Zhang, XiaoFeng Wang, Vincent Bindschaedler, Haixu Tang, and Carl A Gunter. 2017. Leaky cauldron on the dark land: Understanding memory side-channel hazards in SGX. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*. 2421–2434.
- [144] Yao Wang, Andrew Ferraiuolo, and G Edward Suh. 2014. Timing channel protection for a shared memory controller. In *2014 IEEE 20th International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 225–236.
- [145] Yingchen Wang, Riccardo Paccagnella, Zhao Gang, Willy R Vasquez, David Kohlbrenner, Hovav Shacham, and Christopher W Fletcher. 2024. GPU. zip: On the side-channel implications of hardware-based graphical data compression. In *2024 IEEE Symposium on Security and Privacy (SP)*. 84–84.
- [146] Zhenghong Wang and Ruby B Lee. 2007. New cache designs for thwarting software cache-based side channel attacks. In *Proceedings of the 34th Annual International Symposium on Computer Architecture*. 494–505.
- [147] Zhendong Wang, Rujia Wang, Zihang Jiang, Xulong Tang, Shouyi Yin, and Yang Hu. 2021. Towards a secure integrated heterogeneous platform via cooperative CPU/GPU encryption. In *2021 IEEE 30th Asian Test Symposium (ATS)*. IEEE, 115–120.
- [148] Jean-Luc Watson, Sameer Wagh, and Raluca Ada Popa. 2022. Piranha: A GPU platform for secure computation. In *31st USENIX Security Symposium (USENIX Security 22)*. 827–844.
- [149] Junyi Wei, Yicheng Zhang, Zhe Zhou, Zhou Li, and Mohammad Abdullah Al Faruque. 2020. Leaky dnn: Stealing deep-learning model secret with gpu context-switching side-channel. In *2020 50th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. IEEE, 125–137.
- [150] Luca Wilke, Jan Wichelmann, Anja Rabich, and Thomas Eisenbarth. 2024. SEV-step a single-stepping framework for AMD-SEV. *IACR Transactions on Cryptographic Hardware and Embedded Systems* 2024, 1 (2024), 180–206.
- [151] Wenjie Xiong and Jakub Szefer. 2021. Survey of transient execution attacks and their mitigations. *ACM Computing Surveys (CSUR)* 54, 3 (2021), 1–36.
- [152] Yuanzhong Xu, Weidong Cui, and Marcus Peinado. 2015. Controlled-channel attacks: Deterministic side channels for untrusted operating systems. In *2015 IEEE Symposium on Security and Privacy*. IEEE, 640–656.
- [153] Boyuan Yang, Ruirong Chen, Kai Huang, Jun Yang, and Wei Gao. 2022. Eavesdropping user credentials via GPU side channels on smartphones. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. 285–299.
- [154] Ardhi Wiratama Baskara Yudha, Jake Meyer, Shougang Yuan, Huiyang Zhou, and Yan Solihin. 2022. LITE: A low-cost practical inter-operable GPU TEE. In *Proceedings of the 36th ACM International Conference on Supercomputing*. 1–13.
- [155] Zihao Zhan, Zhenkai Zhang, Sisheng Liang, Fan Yao, and Xenofon Koutsoukos. 2022. Graphics peeping unit: Exploiting em side-channel information of gpus to eavesdrop on your neighbors. In *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, 1440–1457.
- [156] Ning Zhang, He Sun, Kun Sun, Wenjing Lou, and Y Thomas Hou. 2016. CacheKit: Evading memory introspection using cache incoherence. In *2016 IEEE European Symposium on Security and Privacy (EuroS&P)*. IEEE, 337–352.
- [157] Ning Zhang, Kun Sun, Deborah Shands, Wenjing Lou, and Y. Thomas Hou. 2016. Truspy: cache side-channel information leakage from the secure world on ARM devices. Retrieved from <https://eprint.iacr.org/2016/421>
- [158] Ruiyi Zhang, CISPA Helmholtz Center, Lukas Gerlach, Daniel Weber, Lorenz Hetterich, Youheng Lü, Andreas Kogler, and Michael Schwarz. 2024. CacheWarp: Software-based fault injection using selective state reset. In *Proceedings of the 33rd USENIX Security Symposium*. USENIX Association, 1135–1151.
- [159] Zhenkai Zhang, Tyler Allen, Fan Yao, Xing Gao, and Rong Ge. 2023. Tunnel for bootlegging: Fully reverse-engineering GPU TLBs for challenging isolation guarantees of NVIDIA MIG. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*. 960–974.
- [160] Wu Zhenyu, Xu Zhang, and H Wang. 2012. Whispers in the hyper-space: High-speed covert channel attacks in the cloud. In *USENIX Security Symposium*. 159–173.

- [161] Zhe Zhou, Wenrui Diao, Xiangyu Liu, Zhou Li, Kehuan Zhang, and Rui Liu. 2016. Vulnerable gpu memory management: Towards recovering raw data from gpu. arXiv:1605.06610. Retrieved from <https://arxiv.org/abs/1605.06610>
- [162] Jianping Zhu, Rui Hou, XiaoFeng Wang, Wenhao Wang, Jiangfeng Cao, Boyan Zhao, Zhongpu Wang, Yuhui Zhang, Jiameng Ying, Lixin Zhang, et al. 2020. Enabling rack-scale confidential computing using heterogeneous trusted execution environment. In *2020 IEEE Symposium on Security and Privacy (SP)*. IEEE, 1450–1465.
- [163] Yuankun Zhu, Yueqiang Cheng, Husheng Zhou, and Yantao Lu. 2021. Hermes attack: Steal DNN models with lossless inference accuracy. In *30th USENIX Security Symposium (USENIX Security 21)*.

Received 3 September 2024; revised 11 June 2025; accepted 19 January 2026